



Universidad de Cuenca

Facultad de Ingeniería
Ingeniería en Telecomunicaciones

Microprocesadores, Microcontroladores y Sistemas Embebidos

Kenneth S. Palacio Baus
kenneth.palacio@ucuenca.edu.ec

Jaime Patricio Chiqui Chiqui
jpatricio.chiquic@ucuenca.edu.ec

Práctica 4

PIC18F4550 - Manejo de Teclado Matricial 4x4 y Control de Microondas

- **Valoración:** 10 puntos.
- **Tipo de Trabajo:** Trabajo práctico e informe individual.
- **Objetivos:** Mediante la presente práctica el estudiante aprenderá a:
 - Implementar la lectura de un teclado matricial 4x4 de 16 teclas.
 - Visualizar datos ingresados mediante displays de 7 segmentos multiplexados.
 - Implementar un cronómetro descendente para simular un horno microondas.
 - Diseñar un sistema con máquina de estados y bloqueo de funciones (Modo Niños).
 - Amplificar salidas de bajo nivel del microcontrolador mediante transistores.
- **Recursos:** Como base de esta practica, utilizaremos MPLAB X y la hoja de datos del microcontrolador PIC18F4550.
- **Instrucciones:** Para obtener una calificación en la presente práctica, cada estudiante deberá entregar un informe escrito según la estructura que se menciona más adelante. No olvide que puede contactar al profesor vía correo electrónico en caso de que necesite asistencia adicional.
 - Envíe su trabajo mediante la plataforma e-nova.
 - El nombre del archivo de su informe debe tener el formato:
ApellidoNombre_Pract04.MicroCon.M26.pdf
 - Si su envío no cumple con el nombre de archivo y la fecha de entrega establecida, no recibirá calificación.

1. Materiales y Equipos

Para el desarrollo de la presente práctica, se utilizaron los siguientes recursos y componentes electrónicos:

- **Software:** MPLAB X IDE v5.x50, compilador XC8 C.
- **Microcontrolador:** PIC18F4550 de Microchip con cristal de cuarzo 20 MHz.
- **Teclado Matricial:** 16 teclas en configuración 4x4 (4 filas, 4 columnas).
- **Displays:** 4 displays de 7 segmentos, cátodo común (Modelo 5641AS).
- **Transistores:**
 - 4 transistores NPN (2N3904) para multiplexación de displays.
 - 1 transistor NPN (2N2222) para control del buzzer.
- **Resistencias:**
 - 14 resistencias de 1 k Ω (displays y control de transistores + buzzer).
 - 7 resistencias de 10 k Ω (4 pull-up para columnas del teclado).
- **Oscilador:** Cristal de cuarzo de 20 MHz (XTAL).
- **Capacitores:** 2 cerámicos de 22 pF (para el cristal).
- **Programador:** *PICkit 3* con su respectivo cable USB.
- **Espadín:** Conector de 6 pines tipo espadín (peineta).
- **Otros:** Protoboard, cables de conexión calibre 22 (o jumpers).
- **Otros componentes:** Buzzer, LED, protoboard.

2. Marco Teórico

2.1. Teclado Matricial 4x4

Un teclado matricial es un dispositivo que permite la entrada de múltiples botones usando el mínimo número de pines del microcontrolador. En lugar de conectar cada botón a un pin individual (lo que requeriría 16 pines), se utiliza una configuración de matriz que solo necesita 8 pines: 4 para filas (outputs) y 4 para columnas (inputs).

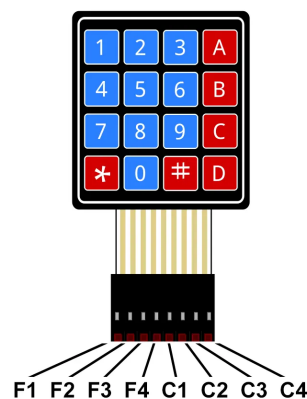


Figura 1: Teclado Matricial 4*4

2.1.1. Principio de Funcionamiento

El principio de operación se basa en la siguiente estrategia:

1. **Activación secuencial de filas:** El microcontrolador pone a cero (0V) una fila a la vez, mientras mantiene las otras filas en nivel alto (+5V).
2. **Lectura de columnas:** Se leen los cuatro pines de las columnas para detectar cuál está en nivel bajo (indicando que una tecla en esa fila fue presionada).
3. **Identificación de la tecla:** La intersección fila-columna identifica unívocamente cuál tecla se presionó.
4. **Tiempo de estabilización:** Se espera 5 ms antes de leer la siguiente fila para evitar ruido.
5. **Antirebote:** Se espera 50 ms adicionales para asegurar que el contacto sea estable.

2.1.2. Distribución de Pines

En el PIC18F4550, el teclado se conecta al Puerto B de la siguiente manera:

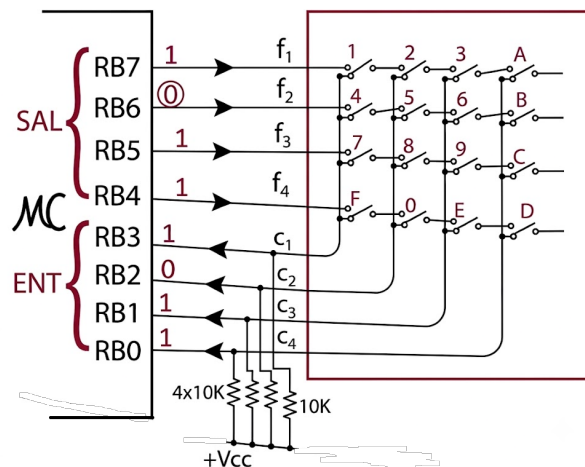


Figura 2: Teclado Matricial 4*4 internamente

2.1.3. Resistencias Pull-up de 10 k

Las resistencias de 10 k conectadas a las columnas (RB0:RB3) tienen funciones críticas:

- **Mantener nivel alto por defecto:** Cuando ninguna tecla está presionada, las columnas están "flotantes" (sin conexión definida). Las resistencias pull-up aseguran que estas líneas permanezcan a +5V.
- **Permitir detección de cambios:** Cuando se presiona una tecla, la fila se pone a 0V, tirando la columna correspondiente a tierra (0V) a través del contacto del botón. Esta transición es claramente detectada.
- **Limitar corriente:** Evitan cortocircuitos excesivos cuando una fila está activa. El límite de corriente es:

$$I = \frac{5V}{10k\Omega} = 0,5 \text{ mA}$$

Completamente seguro para el PIC18F4550.

- **Proteger el microcontrolador:** Las resistencias actúan como divisores de voltaje y limitadores de corriente, protegiendo los pines de entrada.

¿Pueden eliminarse? No. Sin las resistencias pull-up, los pines de entrada RB0:RB3 estarían flotantes, causando lecturas erróneas e impredecibles. El teclado sería completamente inestable. Aunque si **no se desea poner estas resistencias físicamente se puede hacer mediante el mplab habilitando que de RB0 a RB3 sean entradas con resistencias pull up**, mediante el siguiente código:

```
1 INTCON2bits.RBPU = 0;
```

Listing 1: Resist. Pull-Up internas PIC(NO físicas)

2.2. Transistor 2N2222 para Control de Buzzer

El transistor NPN 2N2222 se utiliza para amplificar la corriente de salida de un pin del microcontrolador (máx 25 mA) y controlar un buzzer que requiere mayor corriente.

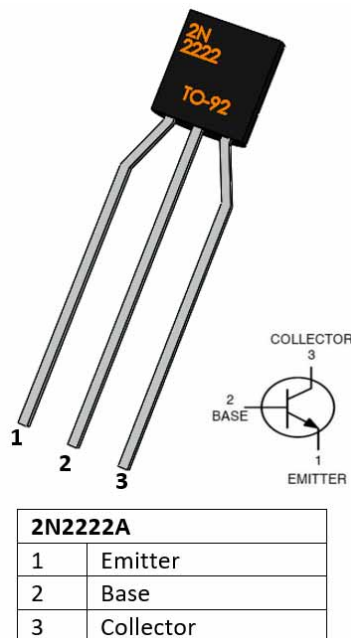


Figura 3: Transistor 2N2222

2.2.1. Características del 2N2222

- **Tipo:** BJT (Bipolar Junction Transistor) NPN.
- **Voltaje máximo Vce:** 30 V.
- **Corriente máxima Ic:** 500 mA.
- **Ganancia (hFE):** 100–300 (típicamente 150).
- **Encapsulado:** TO-92 (3 pines: Base, Colector, Emisor).
- **Aplicaciones:** Amplificación de señales, conmutación de cargas.

2.2.2. Circuito de Activación

El transistor 2N2222 amplifica la corriente del pin RA0 del microcontrolador para accionar el buzzer:

- Entrada: Pin RA0 del microcontrolador (5V cuando activo)
- Resistencia de base: 1 k para limitar corriente de base
- Buzzer conectado entre +5V y colector del transistor
- Emisor del transistor a GND

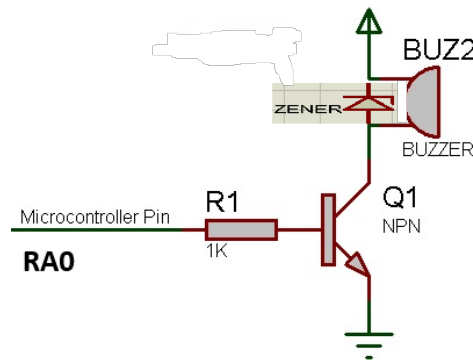


Figura 4: Conexión del Buzzer (Diodo zener opcional)

2.3. Multiplexación de Displays de 7 Segmentos

La multiplexación permite controlar 4 displays usando solo $7 + 4 = 11$ pines del microcontrolador. Se enciende un display a la vez muy rápidamente, apoyándose en la persistencia de visión del ojo humano.

2.3.1. Timer0 y Frecuencia de Refresco

Para evitar parpadeo, se requiere una frecuencia mínima de 50 Hz. En esta práctica:

- **Intervalo de interrupción:** 5 ms por display.
- **Ciclo completo:** $4 \text{ displays} \times 5 \text{ ms} = 20 \text{ ms}$
- **Frecuencia de refresco:** $f = \frac{1}{0,020s} = 50 \text{ Hz}$

2.3.2. Cálculo del Preescalador del TMR0

Para un intervalo de 5 ms con $F_{osc} = 48 \text{ MHz}$:

- Ciclo de máquina: $T_{cy} = \frac{4}{48 \text{ MHz}} = 83,33 \text{ ns}$
- Con preescalador 1:8: $T_{tick} = 83,33 \text{ ns} \times 8 = 666,67 \text{ ns}$
- Cuentas necesarias: $N = \frac{5 \text{ ms}}{666,67 \text{ ns}} = 7500$
- Valor de precarga (16-bit):

$$\text{Precarga} = 65536 - 7500 = 58036 = 0xE2B4$$

- Por lo tanto: $TMR0H = 0xE2$, $TMR0L = 0xB4$

3. Diseño del Sistema Microcontrolado

3.1. Plataforma de Hardware

El circuito implementado en esta práctica consta de:

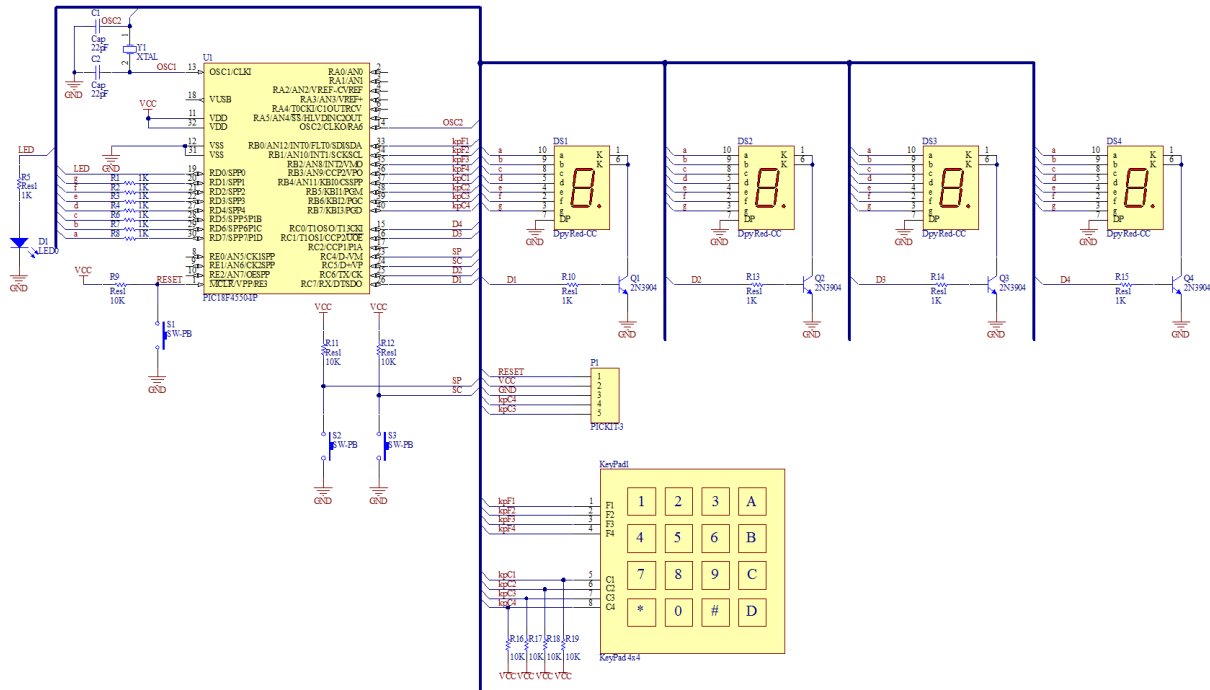


Figura 5: Circuito para multiplexación de displays 7 segmentos: PIC18F4550

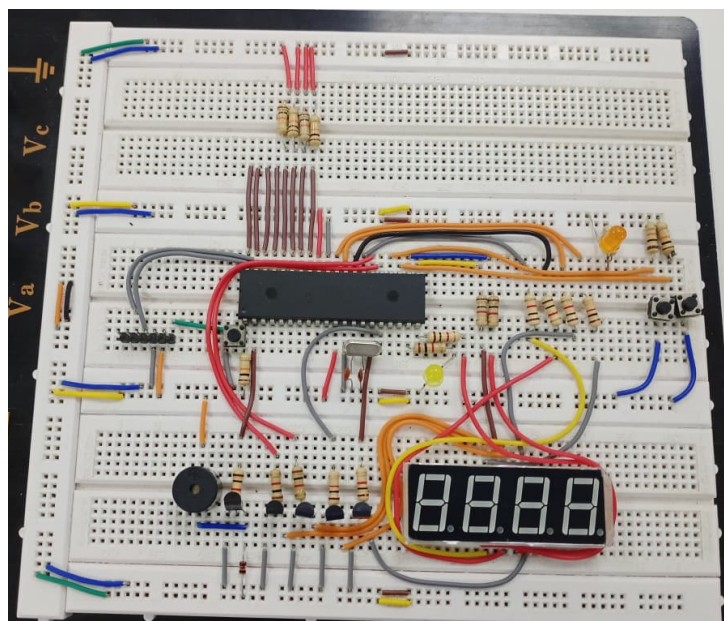


Figura 6: Implementación física del circuito en protoboard

3.1.1. Descripción de Bloques

- **Control de Segmentos (Puerto D):** Conexión a los pines del display mediante resistencias de $1k\Omega$ para limitar la corriente a valores seguros ($< 20\text{ mA}$) y proteger los LEDs.

- **Multiplexación (Puerto C):** Cuatro transistores NPN controlados por el microcontrolador. Al recibir un nivel alto, conectan el cátodo común del display correspondiente a GND, activando un solo dígito a la vez.
- **Indicador de Estado (RD0):** LED con resistencia limitadora de $1k\Omega$ que indicara el magnetron funcionando o apagado de un microondas.

Descripción General

El sistema completo consta de:

1. **Entrada:** Teclado matricial 4x4 (16 teclas).
2. **Procesamiento:** PIC18F4550 con máquina de estados multinivel.
3. **Salida:**
 - 4 displays de 7 segmentos multiplexados
 - LED de estado en RD0
 - Buzzer controlado por transistor en RA0

3.2. Desarrollo del Programa en Lenguaje C/C++

Se desarrolló un programa en lenguaje C/C++ mediante el compilador XC8 en MPLAB X IDE que cumple con las siguientes condiciones de inicio:

3.3. Implemente una subrutina de inicialización con el LED en RD0: 10 pulsos en intervalos de 50ms y luego permanecer encendido. Este LED debe permanecer encendido permanentemente.

El LED en RD0 implementa la siguiente secuencia de inicialización:

1. **10 pulsos con intervalo de 50 ms:** El LED parpadea encendido/apagado 10 veces consecutivas, con un período de 50 ms para cada estado (50 ms ON, 50 ms OFF).
2. **Permanencia encendida:** Después de completar los 10 pulsos, el LED permanece encendido de forma permanente durante toda la operación del sistema, indicando que el microcontrolador está funcionando correctamente.

Implementación en código:

```

1 // INICIO - LED RD0
2 for(int i = 0; i < 10; i++){
3     LATDbits.LATD0 = !LATDbits.LATD0; // Toggle LED (alterna estado)
4     // Delay bloqueante: L = entero largo (desborde)
5     for(long d = 0; d < 43000L; d++); // ~50ms a 48MHz
6 }
7 LATDbits.LATD0 = 1; // LED permanece encendido

```

Listing 2: Secuencia de inicialización del LED RD0

Explicación:

- `LATDbits.LATD0 = !LATDbits.LATD0;` Invierte el estado del LED en cada iteración.
- `for(long d = 0; d < 43000L; d++);` Bucle bloqueante que genera un retardo de aproximadamente 50 ms a 48 MHz.
- La secuencia de 10 parpadeos requiere 1 segundo total (10×100 ms).
- Al final, `LATDbits.LATD0 = 1` asegura que el LED permanezca encendido.

3.4. Determine los códigos para que cada display puede visualizar los números del 0 al 9, en adición a las letras O, n, F,.

Cada display de 7 segmentos requiere un código binario específico que indica qué segmentos deben estar encendidos. La disposición de segmentos es: **a, b, c, d, e, f, g, dp** (punto decimal).

Tabla completa de códigos (Cátodo Común):

Cuadro 1: Códigos hexadecimales para números 0-9 y letras O, n, F (Cátodo Común)

Carácter	Binario	Hexadecimal	Descripción
0	11111100	0xFC	Todos excepto g
1	01100000	0x60	Solo b, c
2	11011010	0xDA	a, b, d, e, g
3	11110010	0xF2	a, b, c, d, g
4	01100110	0x66	b, c, f, g
5	10110110	0xB6	a, c, d, f, g
6	10111110	0xBE	a, c, d, e, f, g
7	11100000	0xE0	a, b, c
8	11111110	0xFE	Todos los segmentos
9	11110110	0xF6	a, b, c, d, f, g
O	10111110	0xBE	Igual a 0
n	00101010	0x2A	c, d, g
F	10001110	0x8E	a, e, f, g

Implementación en código C:

```
1 //Codigo de los numeros 0-9 (c todo com n)
2 const char codigosNum[12] = {
3     0b11111100, // 0
4     0b01100000, // 1
5     0b11011010, // 2
6     0b11110010, // 3
7     0b01100110, // 4
8     0b10110110, // 5
9     0b10111110, // 6
10    0b11100000, // 7
11    0b11111110, // 8
12    0b11110110, // 9
13    0b00101010, // n
14    0b10001110 // F
15 };
16 // Funci n para mostrar d gito en un display
17 void MostrarDigito(char digito) {
18     if(digito >= 0 && digito <= 11) {
19         LATD = codigosNum[digito]; // Env a c digo al puerto
20     }
21 }
```

Listing 3: Array de códigos para números y letras

Validación de códigos:

Cada código fue verificado visualmente en el hardware durante las pruebas. El orden de los bits corresponde directamente a los pines del Puerto D del PIC18F4550 (RD0=dp = led, RD1=g, RD2=f, RD3=e, RD4=d, RD5=c, RD6=b, RD7=a).

4. Partes de la Práctica

4.1. Parte 1: Teclado Matricial + Multiplexación de Displays

4.1.1. Descripción

Se implementa la lectura completa del teclado matricial 4x4 y la visualización de dígitos ingresados en los displays de 7 segmentos. Los números se desplazan de derecha a izquierda (como calculadora). El buzzer proporciona retroalimentación audible cada vez que se presiona una tecla numérica.

4.1.2. Funcionamiento Detallado

1. Inicialización:

- LED RD0 parpadea 10 veces (50 ms c/u) y luego permanece encendido.
- Displays muestran 0000.
- TMR0 configurado para interrupciones cada 5 ms (precarga 0xE2B4).

2. Escaneo de teclado: En cada iteración del bucle principal se llama a LeerTeclado(), que activa una fila a la vez y lee las columnas.

```
1 char LeerTeclado(void) {
2     char tecla = 0xFF; // 0xFF: Nada presionado
3
4     // -----
5     LATB = 0b01110000; // Fila 1
6     DelayXms(5); // Tiempo espera
7     switch (PORTB) {
8         // f1_input: 0111 f1_out: xxxx
9         case 0b01110111: tecla = 1; break;
10        case 0b01111011: tecla = 2; break;
11        case 0b01111101: tecla = 3; break;
12        case 0b01111110: tecla = 10; break; // Tecla 'A'
13    }
14
15    // -----
16    LATB = 0b10110000; // Fila 2:
17    DelayXms(5);
18    switch (PORTB) {
19        // f2_input: 1011 f2_out: xxxx
20        case 0b10110111: tecla = 4; break;
21        case 0b10111011: tecla = 5; break;
22        case 0b10111101: tecla = 6; break;
23        case 0b10111110: tecla = 11; break; // Tecla 'B'
24    }
25    // -----
26    LATB = 0b11010000; // Fila 3
27    DelayXms(5);
28    switch (PORTB) {
29        // f3_input: 1101 f3_out: xxxx
30        case 0b11010111: tecla = 7; break;
31        case 0b11011011: tecla = 8; break;
32        case 0b11011101: tecla = 9; break;
33        case 0b11011110: tecla = 12; break; // Tecla 'C'
34    }
35    // -----
36    LATB = 0b11100000; // Fila 4:
37    DelayXms(5);
38    switch (PORTB) {
39        // f4_input: 1110 f4_out: xxxx
```

```

40     case 0b11100111: tecla = 14; break; // Tecla '*'
41     case 0b11101011: tecla = 0; break;  // 0
42     case 0b11101101: tecla = 15; break; // Tecla '#'
43     case 0b11101110: tecla = 13; break; // Tecla 'D'
44 }
45 // Anti-rebote de 50ms
46 if (tecla != 0xFF) { // presiona algo
47     DelayXms(50);
48     return tecla;
49 }
50 return tecla; //tecla ingresada
51 }

```

Listing 4: Leer Teclado 4*4 — Parte 1

3. Ingreso de números: Solo se aceptan dígitos 0-9. Al presionar una tecla válida:

- Buzzer suena 100 ms (LATA0 alto, transistor en saturación).

```

1 void Buzzer(void) { // Buzzer pin 2 pic
2     LATAbits.LATA0 = 1; // ON buzzer
3     DelayXms(100); // Delay 100ms
4     LATAbits.LATA0 = 0; // OFF buzzer
5 }
6

```

Listing 5: Buzzer 100 ms — Parte 1

- El nuevo dígito ingresa por la derecha (D4) y los anteriores se desplazan a la izquierda: D1←D2, D2←D3, D3←D4.

4. Teclas A, B, C, D, *, #: Son ignoradas en esta parte.

5. Antirebote: Estabilización de fila (5 ms) + confirmación de presión (50 ms adicionales) + espera a soltar (while(LeerTeclado() != 0xFF)).

4.1.3. Código Desplazamiento y lectura del Teclado — Parte 1

```

1 void Desp_izquierda(char tecla_ingresada){
2     digitos[0] = digitos[1]; // D1 <- D2
3     digitos[1] = digitos[2]; // D2 <- D3
4     digitos[2] = digitos[3]; // D3 <- D4
5     digitos[3] = tecla_ingresada; // D4 = nueva tecla
6     ActualizarDisplays();
7 }
8
9 // En el bucle principal:
10 while(1){
11     valor_tecla = LeerTeclado();
12     if(valor_tecla != 0xFF){
13         if(valor_tecla <= 9){ // Solo digitos 0-9
14             Desp_izquierda(valor_tecla);
15             Buzzer(); // Buzzer 100 ms
16         }
17         while(LeerTeclado() != 0xFF); // Espera a soltar
18     }
19 }

```

Listing 6: Desplazamiento y actualización de displays — Parte 1

4.1.4. Tabla de Comportamiento — Parte 1

Cuadro 2: Ejemplo de desplazamiento de dígitos al ingresar teclas

Tecla Presionada	Display	Buzzer
(Inicio)	0000	—
4	0004	100 ms
3	0043	100 ms
1	0431	100 ms
5	4315	100 ms
7	3157	100 ms
A	(rechazada)	—
.	.	.
.	.	.

4.2. Parte 2: Configuración del Reloj (Modo Setup)

4.2.1. Descripción

Se implementa un reloj digital configurable en formato HH:MM. El display parpadea a 1 Hz indicando que la hora aún no está establecida. Mediante las teclas A, B, C y D el usuario configura y confirma la hora inicial.

4.2.2. Funcionamiento Detallado

1. **Estado inicial:** Display muestra 00:00 parpadeando a 1 Hz. LED RD0 también parpadea.
2. **Tecla A — Activar configuración:**
 - Display muestra `rEL` durante 2 segundos con buzzer.
 - Regresa a 00:00 listo para ajuste (`reloj_activo = 0`).
3. **Tecla B — Incrementar minutos:**
 - Pulsación única: +1 minuto.
 - Sostenida (> 20 iteraciones): incremento rápido cada 3 ciclos (≈ 210 ms).
 - Desbordamiento: 59 $\rightarrow$ 0 e incrementa horas.
4. **Tecla C — Decrementar minutos:**
 - Pulsación única: -1 minuto.
 - Sostenida: decremento rápido.
 - Subdesbordamiento: 00 $\rightarrow$ 59, hora -1; si hora es 0, salta a 23.
5. **Tecla D — Confirmar hora:**
 - Activa `reloj_activo = 1`, inicia el conteo del reloj.
 - LED RD0 permanece encendido durante la operación.

4.2.3. Fragmento de Código — Parte 2

```
1 // Parpadeo a 1 Hz (cada 500 ms = 200 interrupciones de 2.5 ms)
2 if(cntISR >= 200){
3     cntISR -= 200;
4     MedioSecCount = !MedioSecCount;
5     LATDbits.LATD0 = !LATDbits.LATD0; // LED parpadea
```

```

6
7     if(!reloj_activo){
8         if(MedioSecCount == 0){
9             datos[0] = datos[1] = datos[2] = datos[3] = 0x00; // OFF
10        } else {
11            ActualizarDisplays(); // Muestra HH:MM
12        }
13    }
14}
15tecla = LeerTeclado();
16// ---- Tecla A: rEL * 2 seg + 0000
17if(tecla == 10 && !reloj_activo){
18    btn_A_cnt++; // Antirebote
19    if(btn_A_cnt == 1){
20        Buzzer();
21        modo_display = 1; // rEL
22        ActualizarDisplays();
23        DelayXms(2000); // rEL * 2 seg
24        modo_display = 0; Hora = 0; Minutos = 0; //0000
25        ActualizarDisplays();
26        ing_btn_A = 1;
27    }
28} else btn_A_cnt = 0; // Setear Antirebote
29
30// Tecla B: incrementar con aceleracion sostenida
31if(tecla == 11 && !reloj_activo && ing_btn_A){
32    btn_B_cnt++;
33    if(btn_B_cnt == 1){ Buzzer(); IncrementarTiempo(); }
34    if(btn_B_cnt > 20 && (btn_B_cnt % 3) == 0) IncrementarTiempo();
35} else { btn_B_cnt = 0; }
36/* -----
37    * t_esp=20 y velo=3 Para inc./dec. al pulsar
38    * 1 while = (3 filas teclado * 5ms) + 50ms delay + 5ms = 70ms.
39    * * 20 vueltas * 70ms = 1400ms = 1.4 seg. // modo presionado auto.
40    * * % 3 : (3 vueltas * 70ms = 210ms).
41    * velocidad cambia el tiempo cada 210ms.
42    * ----- */
43
44// ---- Tecla C: Decrementar Tiempo ----
45if(tecla == 12 && !reloj_activo && ing_btn_A){
46    btn_C_cnt++; // Antirebote
47    if(btn_C_cnt == 1){
48        Buzzer();
49        DecrementarTiempo();
50    }
51    if(btn_C_cnt > 20 && (btn_C_cnt % 3) == 0){
52        DecrementarTiempo();
53    }
54} else btn_C_cnt = 0; // set Antirebote
55// Tecla D: fijar hora
56if(tecla == 13 && !reloj_activo && ing_btn_A){
57    btn_D_cnt++;
58    if(btn_D_cnt == 1){
59        Buzzer();
60        reloj_activo = 1;
61        cuenta_Minutos = 0;
62        cntISR = 0;
63        ActualizarDisplays();
64    }
65} else { btn_D_cnt = 0; }

```

Listing 7: Lógica de incremento/decremento y parpadeo — Parte 2

4.3. Parte 3: Operación del Microondas (Cronómetro Descendente)

4.3.1. Descripción

Una vez configurada la hora o no (Parte 2), ingresado un tiempo directamente con las teclas numéricas (Parte 1) o con la tecla D (+30 seg), el sistema inicia un cronómetro descendente en formato MM:SS. El LED RD0 permanece encendido mientras el magnetrón opera.

4.3.2. Funcionamiento Detallado

1. **Inicio de operación:** Se activa presionando D (agrega 30 s si el tiempo es 00:00) o ingresando 4 dígitos numéricos. `op_Micro = 1`, LED RD0 se enciende.

```
1 // Caso B: Reposo (Reloj) + D, inicia con 30s
2     else if (reloj_activo && !op_Micro) {
3         op_Micro = 1;
4         ing_micro = 0;
5         if (Min_Micro == 0 && Seg_Micro == 0) {
6             Seg_Micro = 30; //0030
7         }
8         LATDbits.LATD0 = 1; // LED RD0 ON (motor y magnetron)
9     }
```

Listing 8: — Parte 3

2. **Conteo regresivo:** Cada 500 ms (200 ticks de 2.5 ms), si `MedioSecCount == 0`, se decrementa el contador:
 - Si `Seg_Micro > 0`: `Seg_Micro--`.
 - Si `Seg_Micro == 0` y `Min_Micro > 0`: `Min_Micro--`, `Seg_Micro = 59`.
 - Si ambos llegan a 0: fin de operación.
3. **Sumar 30 segundos (tecla D durante operación):** Añade 30 s al tiempo restante, con ajuste de carry a minutos.

```
1 } // Caso C: microondas FUNCIONANDO, +30S
2 else if (op_Micro) {
3     Seg_Micro += 30;
4     if (Seg_Micro >= 60) {
5         Seg_Micro -= 60;
6         Min_Micro++;
7     }
8 }
9 ActualizarDisplays();
```

Listing 9: — Parte 3

- **Fin de operación:** Display muestra 00:00, buzzer suena 3 veces, LED apaga, sistema regresa a mostrar reloj.

```
1 LimpiarMicroondas();
2 reloj_activo = 1;
3
4 for(unsigned char b = 0; b < 3; b++) {
5     Buzzer(); //
6     DelayXms(1000);
7 }
```

Listing 10: — Parte 3

4.3.3. Código — Parte 3

```
1 // Decremento cada 1 seg (cada 2 ticks de 500ms)
2 if(op_Micro && MedioSecCount == 0){
3     if(puerta_abierta){
4         DetenerMicroondas(); // Seguridad: puerta abierta
5     }
6     else if(Min_Micro == 0 && Seg_Micro == 0){
7         LimpiarMicroondas(); // Fin: 00:00 alcanzado
8         reloj_activo = 1;
9         for(unsigned char b = 0; b < 3; b++){
10             Buzzer(); DelayXms(1000);
11         }
12     }
13     else if(Seg_Micro == 0){
14         Min_Micro--; // Borrow de minutos
15         Seg_Micro = 59;
16     }
17     else {
18         Seg_Micro--; // Decremento normal
19     }
20 }
21
22 // Tecla D: +30 segundos durante operacion
23 if(tecla == 13 && op_Micro){
24     Seg_Micro += 30;
25     if(Seg_Micro >= 60){ Seg_Micro -= 60; Min_Micro++; }
26     ActualizarDisplays();
27 }
28
29 // Tecla *: Pausa / Clear
30 if(tecla == 14 && !bloqueo_ninos){
31     if(op_Micro){
32         DetenerMicroondas(); // 1ra pulsacion: PAUSA
33     } else {
34         LimpiarMicroondas(); // 2da pulsacion: CLEAR
35     }
36 }
```

Listing 11: Cronómetro descendente y control del magnetron — Parte 3

4.4. Parte 4: Modo Niños — Bloqueo de Teclado

4.4.1. Descripción

Se implementa un modo de bloqueo que inhabilita completamente el teclado para evitar uso accidental. La activación y desactivación se realizan manteniendo presionada la tecla * durante 3 segundos consecutivos, adicionalmente tenemos los modos *Pause/reanudar/clear + modo puerta abierta*, es decir, tenemos las funcionalidades basicas de un microondas real.

Se implemento esto en esta parte ya que es lo esencial que debe tener un microondas, es decir, los metodos de pause/ reanudar/ limpiar ya que es lo que usa comunmente, pero tambien [lo principal es de implementar medidas de seguridad para el usuario](#), como es bloquear este hacia los niños y tambien la puerta abierta.

4.4.2. Activación y Desactivación

- **Activar bloqueo:** Mantener * presionado ≥ 3 segundos (43 ciclos de ≈ 70 ms), con el microondas en reposo (op_Micro = 0).

```
1 if (timer_stop_3s >= 43) {
```

```

2   timer_stop_3s = 0; // Resetea contador
3
4   if (!op_Micro && !ing_micro) {
5       // Si se desactiva el bloqueo, limpiamos los datos viejos
6       bloqueo_ninos = !bloqueo_ninos;
7       if(!bloqueo_ninos) {
8           LimpiarMicroondas();
9       }
10
11       ActualizarDisplays();
12       LATAbits.LATA0 = 1;
13       DelayXms(300);
14       LATAbits.LATA0 = 0;
15
16       conteo_paradas = 0;
17       DelayXms(5); // Evita ghosting
18   }
19 }

```

Listing 12: — Parte 4

- **Desactivar bloqueo:** Mismo procedimiento: mantener * presionado ≥ 3 segundos estando ya bloqueado.
- En ambos casos el buzzer confirma el cambio de estado con un beep de 300 ms.
- Mientras está bloqueado, todas las teclas son ignoradas excepto * (única que puede desbloquear).

```

1 // Bloquear todas las teclas mod n i o s
2 if (bloqueo_ninos && tecla != 14) tecla = 0xFF;

```

Listing 13: — Parte 4

4.4.3. Máquina de Estados — Modo Bloqueo

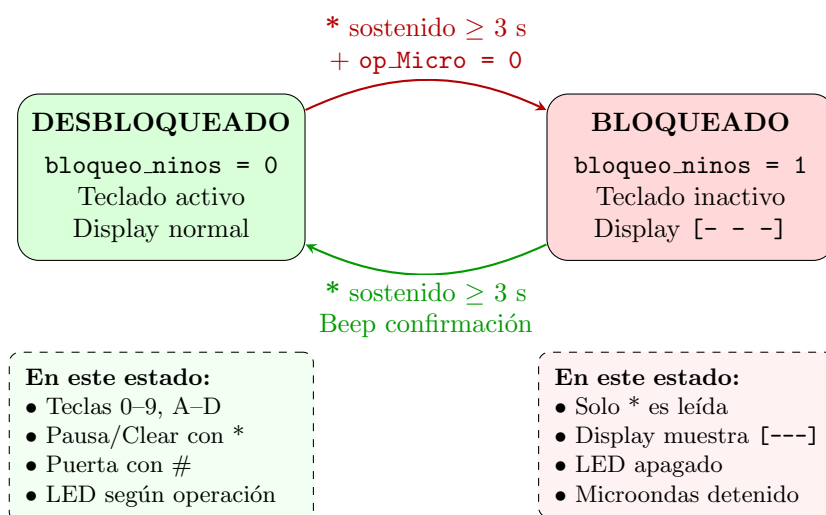


Figura 7: Máquina de estados del Modo Niños (Bloqueo de Teclado)

```

1 if(bloqueo_ninos){
2     datos[0] = 0b10011100; // '[ '
3     datos[1] = 0b10010000; // segmentos a, d
4     datos[2] = 0b10010000; // segmentos a, d

```

```

5   datos[3] = 0b11110000; // ']'
6 }

```

Listing 14: — Parte 4

4.4.4. Adicional Pause/ Reanudar/clear + Puerta abierta

Pausa / clear (tecla *): Primera pulsación pausa (op_Micro = 0, LED apaga). Segunda pulsación limpia el tiempo (LimpiarMicroondas(): Display muestra 0000 y luego HHMM).

```

1  if (timer_stop_3s == 1 && !bloqueo_ninos) {
2      Buzzer();
3      if (op_Micro) { // 1 Pulsaci n cocinando: PAUSA
4          DetenerMicroondas();
5          conteo_paradas = 1;
6      } else {
7          // Pulsaci n en reposo o 2da pulsaci n: CLEAR
8          LimpiarMicroondas();
9          conteo_paradas = 0;
10     }
11     ActualizarDisplays();
12 }
13

```

Listing 15: — Parte 4

Puerta abierta (tecla #): Simula apertura de puerta. Fuerza pausa inmediata por seguridad. Mientras esté abierta, el microondas no puede operar.

```

1  // ---- Tecla #: PUERTA ABIERTA/CERRADA ----
2      if (tecla == 15 && !bloqueo_ninos) {
3          Buzzer();
4          puerta_abierta = !puerta_abierta; // Invierte el estado de la puerta
5
6          if (puerta_abierta) {
7              // Si la puerta se abre, fuerza el apagado
8              DetenerMicroondas();
9              Buzzer(); DelayXms(100); Buzzer();
10         }
11
12         ActualizarDisplays();
13         DelayXms(300); // Delay de antirrebote
14     }

```

Listing 16: — Parte 4

5. Pruebas y Verificaciones

5.1. Parte 1: Teclado y Desplazamiento

Cuadro 3: Pruebas de lectura del teclado y desplazamiento de dígitos

Acción	Display	Buzzer
Encender el sistema	Muestra 0000 (LED RD0 parpadea 10 veces y queda encendido)	Sin activar
Presionar tecla 4	Muestra 0004	Activo 100 ms
Presionar tecla 3	Muestra 0043	Activo 100 ms
Presionar tecla 1	Muestra 0431	Activo 100 ms
Presionar tecla 5	Muestra 4315	Activo 100 ms
Presionar A	Sin cambio (Tecla ignorada)	Sin activar
Presionar *	Sin cambio (Tecla ignorada)	Sin activar
Pulsaciones rápidas sucesivas	Una lectura por tecla	Antirebote
Presionar tecla 5	Muestra 3155	Activo 100 ms
Presionar tecla 5	Muestra 1555	Activo 100 ms
.		..

5.2. Parte 2: Configuración del Reloj

Cuadro 4: Pruebas del modo configuración de reloj HH:MM

Acción	Display	Buzzer
Encender	Display 00:00 parpadeando a 1 Hz (LED RD0 parpadea)	Sin activar
Presionar A	Muestra rEL por 2 s y luego vuelve a 00:00	Activo (2 s)
Presionar B (una vez)	Minutos +1, display 00:01	Sin activar
Mantener B presionado	Incremento automático cada ≈ 210 ms	Sin activar
Presionar C (una vez)	Minutos -1, display 00:00	Sin activar
C desde 00:00	Salta a 23:59 (wrap correcto)	Sin activar
Presionar D	Reloj fijado, inicia conteo (LED RD0 encendido permanente)	Sin activar

5.3. Parte 3: Operación del Microondas

Cuadro 5: Pruebas del cronómetro descendente MM:SS

Acción	Display	Buzzer
Ingresar 0010 y presionar D	Cronómetro cuenta descendente: 00:09, 00:08, ..., 00:00	Sin activar
Durante operación	LED RD0 permanece encendido	Sin activar
En 00:15, presionar D	Salta a 00:45 (+30 s)	Sin activar
Presionar * durante operación	Pausa inmediata (LED RD0 se apaga)	Sin activar
Presionar # durante operación	Puerta abierta: pausa forzada por seguridad	Sin activar
Llegada a 00:00	Vuelve a mostrar el reloj (LED RD0 se apaga)	Suena 3 veces
Presionar D sin tiempo previo	Inicia con 30 s automáticamente	Sin activar

5.4. Parte 4: Modo Niños (Bloqueo)

Cuadro 6: Pruebas del sistema de bloqueo por tecla * sostenida

Acción	Display	Buzzer
Mantener * por ≥ 3 s en reposo	Muestra [---] (teclado bloqueado)	Beep de 300 ms
Bloqueado: presionar cualquier tecla (no *)	Tecla ignorada completamente (sin cambio)	Sin activar
Bloqueado: presionar * brevemente	Sin efecto/cambio (timer no alcanza 43 ciclos)	Sin activar
Bloqueado: mantener * por ≥ 3 s	Desbloqueo, display vuelve a la normalidad (HHMM)	Beep de confirmación
Intentar bloquear con microondas activo	Sistema no permite bloqueo (op_Micro = 1)	Sin activar

6. Códigos Completos y Repositorio

Los códigos completos para las tres partes de la práctica están disponibles en C/C++ implementados con MPLAB X IDE v5.50, así como los videos de funcionamiento, se encuentran disponibles en el repositorio de GitHub:

Archivos de código fuente:

- [Códigos fuente \(Práctica 4\)](#)

Videos de demostración:

- [Video Parte 1 — Teclado matricial y multiplexación de displays](#)
- [Video Parte 2 — Configuración del reloj HH:MM](#)
- [Video Parte 3 y 4 — Cronómetro descendente y Modo Niños](#)

Repositorio principal:

- [Repositorio Principal - Todas las Prácticas](#)

7. Conclusiones y Recomendaciones

7.1. Conclusiones

Con base en la resolución de esta práctica se concluye que:

- La lectura de un teclado matricial 4×4 mediante activación secuencial de filas reduce el uso de pines de 16 a 8, demostrando que la multiplexación matricial es una técnica eficiente y escalable. El algoritmo basado en `switch-case` sobre el valor completo del puerto resultó más robusto frente al ruido que la lectura bit a bit individual.
- Los delays de 5 ms (estabilización de fila) y 50 ms (antirebote) son indispensables para la lectura confiable del teclado. Sin ellos, una sola pulsación física es interpretada como múltiples eventos, corrompiendo la entrada de datos. Estos valores representan un compromiso entre confiabilidad mecánica y velocidad de respuesta del sistema.
- La integración del teclado con la multiplexación de displays mediante la ISR del TMR0 demostró la viabilidad de sistemas multitarea cooperativos en microcontroladores de 8 bits: la ISR gestiona el refresco visual de forma transparente mientras el bucle principal atiende la lógica del microondas sin bloqueos perceptibles.
- El cronómetro descendente basado en la acumulación de ticks de 2.5 ms permite implementar una base de tiempo de 1 segundo precisa. La estrategia de restar 200 al contador en lugar de resetearlo preserva los ticks sobrantes y evita deriva acumulativa en el conteo de tiempo, si se conservara (no restar esos 200) este por cada minuto se aumentaría 3 segundos es decir para el primer minuto debe pasar 63 segundos, para el segundo minuto debería pasar 66 segundos y así sucesivamente pero no estaría bien ya que siempre debería ser 60 seg por cada minuto.
- El transistor NPN 2N2222 como interruptor permite al PIC18F4550 controlar cargas inductivas como el buzzer sin riesgo para sus pines de salida.
- La máquina de estados implementada (reposo, configuración de reloj, operación del microondas, bloqueo) demostró ser una arquitectura de software escalable y mantenible. Cada estado tiene transiciones y comportamientos bien definidos, facilitando la adición del Modo Niños como aporte individual sin modificar la lógica de las partes anteriores.
- La programación en C con XC8 permitió expresar lógica compleja de forma legible y estructurada, aunque la gestión manual de las variables `volatile` y la carga de la ISR requirieron comprensión profunda del hardware para evitar condiciones de carrera entre el bucle principal y las interrupciones.

7.2. Recomendaciones

- **Protección del buzzer:** Agregar un diodo de rueda libre en antiparalelo con el buzzer protege al transistor de los picos de tensión inversa generados al desactivar la carga inductiva, aumentando la vida útil del circuito.

8. Anexos

8.1. Configuración de Bits (Pragma Config)

```
1 // CONFIG1L
2 #pragma config PLLDIV = 5          // PLL Prescaler Selection bits (No prescale
   (4 MHz oscillator input drives PLL directly))
3 #pragma config CPUDIV = OSC1_PLL2 // System Clock Postscaler Selection bits ([
   Primary Oscillator Src: /1][96 MHz PLL Src: /2])
4 #pragma config USBDIV = 1          // USB Clock Selection bit (used in Full-Speed
   USB mode only; UCFG:FSEN = 1) (USB clock source comes directly from the
   primary oscillator block with no postscale)
5
6 // CONFIG1H
7 #pragma config FOSC = HSPLL_HS     // Oscillator Selection bits (HS oscillator,
   PLL enabled (HSPLL))
8 #pragma config FCMEN = OFF         // Fail-Safe Clock Monitor Enable bit (Fail-
   Safe Clock Monitor disabled)
9 #pragma config IESO = OFF          // Internal/External Oscillator Switchover bit
   (Oscillator Switchover mode disabled)
10
11 // CONFIG2L
12 #pragma config PWRT = ON           // Power-up Timer Enable bit (PWRT enabled)
13 #pragma config BOR = ON           // Brown-out Reset Enable bits (Brown-out
   Reset enabled in hardware only (SBOREN is disabled))
14 #pragma config BORV = 3           // Brown-out Reset Voltage bits (Minimum
   setting 2.05V)
15 #pragma config VREGEN = OFF        // USB Voltage Regulator Enable bit (USB
   voltage regulator disabled)
16
17 // CONFIG2H
18 #pragma config WDT = OFF           // Watchdog Timer Enable bit (WDT disabled (
   control is placed on the SWDTEN bit))
19 #pragma config WDTPS = 32768      // Watchdog Timer Postscale Select bits
   (1:32768)
20
21 // CONFIG3H
22 #pragma config CCP2MX = ON         // CCP2 MUX bit (CCP2 input/output is
   multiplexed with RC1)
23 #pragma config PBADEN = OFF        // PORTB A/D Enable bit (PORTB<4:0> pins are
   configured as digital I/O on Reset)
24 #pragma config LPT1OSC = OFF       // Low-Power Timer 1 Oscillator Enable bit (
   Timer1 configured for higher power operation)
25 #pragma config MCLRE = ON          // MCLR Pin Enable bit (MCLR pin enabled; RE3
   input pin disabled)
26
27 // CONFIG4L
28 #pragma config STVREN = ON         // Stack Full/Underflow Reset Enable bit (
   Stack full/underflow will cause Reset)
29 #pragma config LVP = OFF           // Single-Supply ICSP Enable bit (Single-
   Supply ICSP enabled)
30 #pragma config ICPRT = OFF         // Dedicated In-Circuit Debug/Programming Port
   (ICPORT) Enable bit (ICPORT disabled)
31 #pragma config XINST = OFF         // Extended Instruction Set Enable bit (
   Instruction set extension and Indexed Addressing mode disabled (Legacy
   mode))
32 // ** Apagar LVP para no tener corriente parasita *****
33
34 // CONFIG5L
35 #pragma config CP0 = OFF           // Code Protection bit (Block 0 (000800-001
   FFFh) is not code-protected)
36 #pragma config CP1 = OFF           // Code Protection bit (Block 1 (002000-003
   FFFh) is not code-protected)
```

```

37 #pragma config CP2 = OFF           // Code Protection bit (Block 2 (004000-005
    FFFh) is not code-protected)
38 #pragma config CP3 = OFF           // Code Protection bit (Block 3 (006000-007
    FFFh) is not code-protected)
39
40 // CONFIG5H
41 #pragma config CPB = OFF           // Boot Block Code Protection bit (Boot block
    (000000-0007FFh) is not code-protected)
42 #pragma config CPD = OFF           // Data EEPROM Code Protection bit (Data
    EEPROM is not code-protected)
43
44 // CONFIG6L
45 #pragma config WRT0 = OFF           // Write Protection bit (Block 0 (000800-001
    FFFh) is not write-protected)
46 #pragma config WRT1 = OFF           // Write Protection bit (Block 1 (002000-003
    FFFh) is not write-protected)
47 #pragma config WRT2 = OFF           // Write Protection bit (Block 2 (004000-005
    FFFh) is not write-protected)
48 #pragma config WRT3 = OFF           // Write Protection bit (Block 3 (006000-007
    FFFh) is not write-protected)
49
50 // CONFIG6H
51 #pragma config WRTC = OFF           // Configuration Register Write Protection bit
    (Configuration registers (300000-3000FFh) are not write-protected)
52 #pragma config WRTB = OFF           // Boot Block Write Protection bit (Boot block
    (000000-0007FFh) is not write-protected)
53 #pragma config WRTD = OFF           // Data EEPROM Write Protection bit (Data
    EEPROM is not write-protected)
54
55 // CONFIG7L
56 #pragma config EBTR0 = OFF           // Table Read Protection bit (Block 0
    (000800-001FFFh) is not protected from table reads executed in other
    blocks)
57 #pragma config EBTR1 = OFF           // Table Read Protection bit (Block 1
    (002000-003FFFh) is not protected from table reads executed in other
    blocks)
58 #pragma config EBTR2 = OFF           // Table Read Protection bit (Block 2
    (004000-005FFFh) is not protected from table reads executed in other
    blocks)
59 #pragma config EBTR3 = OFF           // Table Read Protection bit (Block 3
    (006000-007FFFh) is not protected from table reads executed in other
    blocks)
60
61 // CONFIG7H
62 #pragma config EBTRB = OFF           // Boot Block Table Read Protection bit (Boot
    block (000000-0007FFh) is not protected from table reads executed in other
    blocks)

```

Listing 17: Pragma Config PIC18F4550

8.2. Códigos Completos de Cada Parte

```

1 /*
2  * File:    Pr3_Codigo_Parte2.c
3  * Author:  JAIME CHIQUI
4  *
5  * Created on 28 de mayo de 2026, 10:42
6  */
7 // PRACTICA 3 - PARTE 1: Teclado Matricial 4x4 y Mult. Displays
8 // simulando el ingreso de datos de una calculadora/microondas
9 // PIC18F4550 Configuration Bit Settings
10

```

```

11
12 #include <xc.h>
13
14 // =====
15 //                               VARIABLES GLOBALES
16 // =====
17 volatile char datos[4]    = {0x00, 0x00, 0x00, 0x00};
18 volatile char multiplex  = 0;
19 volatile char digitos[4]  = {0, 0, 0, 0};
20
21 //Codigo de los numeros 0-9 (c todo com n, [a b c d e f g dp])
22 const char codigosNum[12] = {
23     0b11111100, // 0
24     0b01100000, // 1
25     0b11011010, // 2
26     0b11110010, // 3
27     0b01100110, // 4
28     0b10110110, // 5
29     0b10111110, // 6
30     0b11100000, // 7
31     0b11111110, // 8
32     0b11110110, // 9
33     0b00101010, // n
34     0b10001110  // F
35 };
36
37 // =====
38 //                               DELAYS
39 // =====
40 void Delay1ms(){
41     int veces;
42     for ( veces=0; veces <= 860; veces++ );
43 }
44
45 void DelayXms(int Milis){
46     for(int i=0; i<= Milis; i++){
47         Delay1ms();
48     }
49 }
50
51 // =====
52 //                               ACTUALIZAR DISPLAYS TECLA INGRESADA
53 // =====
54 void ActualizarDisplays(void){
55     datos[0] = codigosNum[(int)digitos[0]]; // D1
56     datos[1] = codigosNum[(int)digitos[1]]; // D2
57     datos[2] = codigosNum[(int)digitos[2]]; // D3
58     datos[3] = codigosNum[(int)digitos[3]]; // D4
59 }
60 // =====
61 //                               DESPLAZAR VALORES A LA IZQUIERDA + TECLA INGRESADA
62 // =====
63 void Desp_izquierda(char tecla_ingresada){
64     digitos[0] = digitos[1]; // D1
65     digitos[1] = digitos[2]; // D2
66     digitos[2] = digitos[3]; // D3
67     digitos[3] = tecla_ingresada; //D4
68     ActualizarDisplays();
69 }
70 // =====
71 //                               BUZZER
72 // =====
73 void Buzzer(void){ // Buzzer pin 2 pic

```

```

74     LATAbits.LATA0 = 1; // ON buzzer
75     DelayXms(100);      // Delay 100ms
76     LATAbits.LATA0 = 0; // OFF buzzer
77 }
78 // =====
79 //             LEER TECLADO MATRICIAL (Filas RB4-RB7, Col RB0-RB3)
80 // =====
81 char LeerTeclado(void){
82     char tecla = 0xFF; // 0xFF: Nada presionado
83
84     // -----
85     // Fila 1: RB7=0, RB6=1, RB5=1, RB4=1 -> 0b01110000
86     // -----
87     LATB = 0b01110000;
88     DelayXms(5); // Tiempo espera
89     switch (PORTB) {
90         // f1_input: 0111 f1_out: xxxx
91         case 0b01110111: tecla = 1; break;
92         case 0b01111011: tecla = 2; break;
93         case 0b01111101: tecla = 3; break;
94         case 0b01111110: tecla = 10; break; // Tecla 'A'
95     }
96     // Anti-rebote de 50ms
97     if(tecla != 0xFF) { // presiona algo
98         DelayXms(50);
99         return tecla;
100     }
101     // -----
102     // Fila 2: RB7=1, RB6=0, RB5=1, RB4=1 -> 0b10110000
103     // -----
104     LATB = 0b10110000;
105     DelayXms(5);
106     switch (PORTB) {
107         // f2_input: 1011 f2_out: xxxx
108         case 0b10110111: tecla = 4; break;
109         case 0b10111011: tecla = 5; break;
110         case 0b10111101: tecla = 6; break;
111         case 0b10111110: tecla = 11; break; // Tecla 'B'
112     }
113     // Anti-rebote de 50ms
114     if(tecla != 0xFF) { // presiona algo
115         DelayXms(50);
116         return tecla;
117     }
118
119     // -----
120     // Fila 3: RB7=1, RB6=1, RB5=0, RB4=1 -> 0b11010000
121     // -----
122     LATB = 0b11010000;
123     DelayXms(5);
124     switch (PORTB) {
125         // f3_input: 1101 f3_out: xxxx
126         case 0b11010111: tecla = 7; break;
127         case 0b11011011: tecla = 8; break;
128         case 0b11011101: tecla = 9; break;
129         case 0b11011110: tecla = 12; break; // Tecla 'C'
130     }
131     // Anti-rebote de 50ms
132     if(tecla != 0xFF) { // presiona algo
133         DelayXms(50);
134         return tecla;
135     }
136 }

```

```

137 // -----
138 // Fila 4: RB7=1, RB6=1, RB5=1, RB4=0 -> 0b11100000
139 // -----
140 LATB = 0b11100000;
141 DelayXms(5);
142 switch (PORTB) {
143     // f4_input: 1110 f4_out: xxxx
144     case 0b11100111: tecla = 14; break; // Tecla '*'
145     case 0b11101011: tecla = 0; break; // 0
146     case 0b11101101: tecla = 15; break; // Tecla '#'
147     case 0b11101110: tecla = 13; break; // Tecla 'D'
148 }
149 // Anti-rebote de 50ms
150 if(tecla != 0xFF) { // presiona algo
151     DelayXms(50);
152     return tecla;
153 }
154 return 0xFF; // Ninguna tecla ingresada
155 }
156 // =====
157 //             ISR ALTA PRIORIDAD - TMRO 16 bits
158 // =====
159 void __interrupt(high_priority) InterrupcionHP(void){
160     if(INTCONbits.TMROIF){
161         // == INTERRUPCIONES DE 5 ms ==
162         // Tiempo total del ciclo = 4 displays * 5 ms = 20 ms
163         // =====
164         // TMRO: 5ms (Fosc=48MHz, 16-bit, Prescaler 1:8)
165         // =====
166         // Tcy = 1 / (48MHz / 4) = 83.33ns
167         // Tick = Tcy * 8 (Prescaler) = 666.67ns
168         // Conteos = 5ms / 666.67ns = 7500
169         // Carga = 65536 - 7500 = 58036 = 0xE2B4
170         TMROH = 0xE2; // Parte alta
171         TMROL = 0xB4; // Parte baja
172
173         // OFF DISPLAY( evita ghosting)
174         LATCbits.LATC7 = 0; // D1
175         LATCbits.LATC6 = 0; // D2
176         LATCbits.LATC1 = 0; // D3
177         LATCbits.LATC0 = 0; // D4
178
179         // Multiplexaci n
180         LATD = (datos[multiplex] & 0xFE) | (LATDbits.LATD0);
181         switch(multiplex){
182             case 0: LATCbits.LATC7 = 1; break;
183             case 1: LATCbits.LATC6 = 1; break;
184             case 2: LATCbits.LATC1 = 1; break;
185             case 3: LATCbits.LATC0 = 1; break;
186         }
187         multiplex++;
188         if(multiplex > 3) multiplex = 0;
189
190         INTCONbits.TMROIF = 0;
191     }
192 }
193
194 // =====
195 //             MAIN
196 // =====
197 void main(void){
198     // --- CONFIGURACI N CR TICA PARA USAR RC4 y RC5 ---
199     UCONbits.USBEN = 0; // Deshabilita el m dulo USB

```



```

200 UCFGbits.UTRDIS = 1; // Deshabilita el transceptor USB (E/S digital)
201 ADCON1 = 0x0F;      // Buzzer en RAO
202 // -----
203
204 // --- Configuraci n de Puertos ---
205 TRISD = 0;
206 TRISCbits.TRISCO = 0;
207 TRISCbits.TRISC1 = 0;
208 TRISCbits.TRISC6 = 0;
209 TRISCbits.TRISC7 = 0;
210 // TRISC |= 0b00110000; // Activar s2 y s3
211
212 // ===== PORT B - Teclado 4*4 Matricial =====
213 // RB0-RB3: Entradas (Columnas), RB4-RB7: Salidas (Filas)
214 TRISB = 0b00001111; // // 0xFF: Nada presionado
215 LATB = 0b11110000;   // // filas en 1
216 // INTCON2bits.RBPUP = 0; // Resist. Pull-Up internas PIC(NO fisicas)
217
218 LATD = 0;
219 LATCbits.LATC7 = 0;
220 LATCbits.LATC6 = 0;
221 LATCbits.LATC1 = 0;
222 LATCbits.LATC0 = 0;
223
224 // Port A para Buzzer
225 TRISAbits.TRISA0 = 0;
226 LATAbits.LATA0 = 0;
227
228 // ==== INTERRUPTACIONES DE 5 ms ====
229 // Tiempo total del ciclo = 4 displays * 5 ms = 20 ms
230 // =====
231 // TMR0: 5ms (Fosc=48MHz, 16-bit, Prescaler 1:8)
232 // =====
233 // Tcy = 1 / (48MHz / 4) = 83.33ns
234 // Tick = Tcy * 8 (Prescaler) = 666.67ns
235 // Conteos = 5ms / 666.67ns = 7500
236 // Carga = 65536 - 7500 = 58036 = 0xE2B4
237 TMR0H = 0xE2; // Parte alta
238 TMR0L = 0xB4; // Parte baja
239 TOCON = 0b00000010;
240
241 // Configuraci n del TOCON:
242 // bit 7: TMR0ON = 0 (Apagado por ahora)
243 // bit 6: T08BIT = 0 (Configurado en 16 bits)
244 // bit 5: TOCS   = 0 (Reloj interno Fosc/4)
245 // bit 4: TOSE   = 0 (No reloj interno)
246 // bit 3: PSA    = 0 (Prescaler activado)
247 // bit 2-0: TOPS = 010 (Prescaler 1:8)
248
249 RCONbits.IPEN      = 1;
250 INTCON2bits.TMR0IP = 1;
251 INTCONbits.TMR0IE  = 1;
252 INTCONbits.TMR0IF  = 0;
253 INTCONbits.GIEH    = 1;
254 INTCONbits.GIEL    = 1;
255 TOCONbits.TMR0ON   = 1;
256
257 // INICIO - LED RDO *****
258 for(int i = 0; i < 10; i++){
259     LATDbits.LATD0 = !LATDbits.LATD0;
260     // delay bloqueante: L = entero largo (desborde)
261     for(long d = 0; d < 43000L; d++); // ~50ms a 48MHz

```

```

262     }
263     LATDbits.LATD0 = 1;
264
265     ActualizarDisplays(); //0000
266     char valor_tecla;
267
268     while(1){
269         valor_tecla = LeerTeclado();
270
271         if(valor_tecla != 0xFF){ // Se ingreso algo
272             // Filtrar que se ingresen solo numeros 0-9
273             if(valor_tecla <= 9){
274                 Desp_izquierda(valor_tecla); // Nuevo valor a la izq.
275                 Buzzer(); // ON: 100ms
276             }
277             while(LeerTeclado() != 0xFF); // Antirebote Teclado 4*4
278         }
279     }
280     return;
281 }

```

Listing 18: Pr4.Partel.c – Teclado + Displays

```

1  /*
2   * File:    Pr4_Parte2.c
3   * Author:  JAIME CHIQUI
4   * Created on 4 de junio de 2026, 9:35
5   */
6  // PRACTICA 3 - PARTE 2: RELOJ HH:MM (Sim Microondas)
7
8  #include <xc.h>
9
10 // =====
11 //                      VARIABLES GLOBALES
12 // =====
13 volatile char datos[4] = {0x00, 0x00, 0x00, 0x00};
14 volatile char multiplex = 0;
15 volatile char digitos[4] = {0, 0, 0, 0};
16
17 // Flags, HH:SS y contadores de ISR
18 volatile unsigned int cntISR = 0; // cuenta int. para 1 s
19 int Hora = 0;
20 int Minutos = 0;
21 char reloj_activo = 0; // 0 = ajuste (parpadea), 1 = fija (reloj)
22 char modo_display = 0; // 1 = rEL
23
24 // Codigo de los numeros 0-9 (c todo com n, [a b c d e f g dp])
25 const char codigosNum[12] = {
26     0b11111100, // 0
27     0b01100000, // 1
28     0b11011010, // 2
29     0b11110010, // 3
30     0b01100110, // 4
31     0b10110110, // 5
32     0b10111110, // 6
33     0b11100000, // 7
34     0b11111110, // 8
35     0b11110110, // 9
36     0b00101010, // n
37     0b10001110 // F
38 };
39 //Mostrar rEL
40 const char rEL[4] = {
41     0b00001010, // D1 'r'

```

```

42     0b10011110, // D2 'E'
43     0b00011100, // D3 'L'
44     0b00000000 // D4 OFF
45 };
46 // =====
47 //                               DELAYS
48 // =====
49 void Delay1ms(){
50     int veces;
51     for ( veces=0; veces <= 860; veces++ );
52 }
53
54 void DelayXms(int Milis){
55     for(int i=0; i<= Milis; i++){
56         Delay1ms();
57     }
58 }
59
60 // =====
61 //                               ACTUALIZAR DISPLAYS TECLA INGRESADA
62 // =====
63 void ActualizarDisplays(void){
64     if (modo_display){ //Mostrar palabra rEL
65         datos[0] = rEL[0]; // D1: r
66         datos[1] = rEL[1]; // D2: E
67         datos[2] = rEL[2]; // D3: L
68         datos[3] = rEL[3]; // D4: OFF
69     }else{
70         datos[0] = codigosNum[Hora / 10]; // D1: decenas Hora
71         datos[1] = codigosNum[Hora % 10]; // D2: unidades Hora
72         datos[2] = codigosNum[Minutos / 10]; // D3: decenas Minutos
73         datos[3] = codigosNum[Minutos % 10]; // D4: unidades Minutos
74
75         // ejemplo: 37 / 10 = 3.7 (int en c: 3)
76         //           37 % 10 = 7
77     }
78 }
79 // =====
80 //                               DESPLAZAR VALORES A LA IZQUIERDA + TECLA INGRESADA
81 // =====
82 void Desp_izquierda(char tecla_ingresada){
83     digitos[0] = digitos[1]; // D1
84     digitos[1] = digitos[2]; // D2
85     digitos[2] = digitos[3]; // D3
86     digitos[3] = tecla_ingresada; //D4
87     ActualizarDisplays();
88 }
89 void IncrementarTiempo(void){
90     Minutos++;
91     if(Minutos > 59){ Minutos = 0; Hora++; }
92     if(Hora > 23) Hora = 0; // Horas
93     ActualizarDisplays();
94 }
95 void DecrementarTiempo(void){
96     if(Minutos == 0){
97         Minutos = 59;
98         if(Hora == 0) Hora = 23; // Horas
99         else Hora--;
100     } else {
101         Minutos--;
102     }
103     ActualizarDisplays();
104 }

```

```

105 // =====
106 //          BUZZER
107 // =====
108 void Buzzer(void){ // Buzzer pin 2 pic
109     LATAbits.LATA0 = 1; // ON buzzer
110     DelayXms(100);      // Delay 100ms
111     LATAbits.LATA0 = 0; // OFF buzzer
112 }
113 // =====
114 //          LEER TECLADO MATRICIAL (Filas RB4-RB7, Col RB0-RB3)
115 // =====
116 char LeerTeclado(void){
117     char tecla = 0xFF; // 0xFF: Nada presionado
118
119     // -----
120     // Fila 1: RB7=0, RB6=1, RB5=1, RB4=1 -> 0b01110000
121     // -----
122     LATB = 0b01110000;
123     DelayXms(5); // Tiempo espera
124     switch (PORTB) {
125         // f1_input: 0111 f1_out: xxxx
126         case 0b01110111: tecla = 1; break;
127         case 0b01111011: tecla = 2; break;
128         case 0b01111101: tecla = 3; break;
129         case 0b01111110: tecla = 10; break; // Tecla 'A'
130     }
131     // Anti-rebote de 50ms
132     if(tecla != 0xFF) { // presiona algo
133         DelayXms(50);
134         return tecla;
135     }
136     // -----
137     // Fila 2: RB7=1, RB6=0, RB5=1, RB4=1 -> 0b10110000
138     // -----
139     LATB = 0b10110000;
140     DelayXms(5);
141     switch (PORTB) {
142         // f2_input: 1011 f2_out: xxxx
143         case 0b10110111: tecla = 4; break;
144         case 0b10111011: tecla = 5; break;
145         case 0b10111101: tecla = 6; break;
146         case 0b10111110: tecla = 11; break; // Tecla 'B'
147     }
148     // Anti-rebote de 50ms
149     if(tecla != 0xFF) { // presiona algo
150         DelayXms(50);
151         return tecla;
152     }
153
154     // -----
155     // Fila 3: RB7=1, RB6=1, RB5=0, RB4=1 -> 0b11010000
156     // -----
157     LATB = 0b11010000;
158     DelayXms(5);
159     switch (PORTB) {
160         // f3_input: 1101 f3_out: xxxx
161         case 0b11010111: tecla = 7; break;
162         case 0b11011011: tecla = 8; break;
163         case 0b11011101: tecla = 9; break;
164         case 0b11011110: tecla = 12; break; // Tecla 'C'
165     }
166     // Anti-rebote de 50ms
167     if(tecla != 0xFF) { // presiona algo

```

```

168     DelayXms(50);
169     return tecla;
170 }
171
172 // -----
173 // Fila 4: RB7=1, RB6=1, RB5=1, RB4=0 -> 0b11100000
174 // -----
175 LATB = 0b11100000;
176 DelayXms(5);
177 switch (PORTB) {
178     // f4_input: 1110 f4_out: xxxx
179     case 0b11100111: tecla = 14; break; // Tecla '*'
180     case 0b11101011: tecla = 0; break; // 0
181     case 0b11101101: tecla = 15; break; // Tecla '#'
182     case 0b11101110: tecla = 13; break; // Tecla 'D'
183 }
184 // Anti-rebote de 50ms
185 if(tecla != 0xFF) { // presiona algo
186     DelayXms(50);
187     return tecla;
188 }
189 return 0xFF; // Ninguna tecla ingresada
190 }
191 // =====
192 //             ISR ALTA PRIORIDAD - TMRO 16 bits
193 // =====
194 void __interrupt(high_priority) InterrupcionHP(void){
195     if(INTCONbits.TMROIF){
196         // ==== INTERRUPTACIONES DE 2.5 ms ====
197         // Tiempo total del ciclo = 4 displays * 2.5 ms = 10 ms
198         // =====
199         // TMRO: 5ms (Fosc=48MHz, 16-bit, Prescaler 1:8)
200         // =====
201         // Tcy = 1 / (48MHz / 4) = 83.33ns
202         // Tick = Tcy * 8 (Prescaler) = 666.67ns
203         // Conteos = 2.5ms / 666.67ns = 3750
204         // Carga = 65536 - 3750 = $61786 = 0xF15A
205         TMROH = 0xF1;
206         TMROL = 0x5A;
207
208         // OFF DISPLAY(evita ghosting)
209         LATCbits.LATC7 = 0; // D1
210         LATCbits.LATC6 = 0; // D2
211         LATCbits.LATC1 = 0; // D3
212         LATCbits.LATC0 = 0; // D4
213
214         // Multiplexaci n
215         LATD = (datos[multiplex] & 0xFE) | (PORTDbits.RD0);
216         switch(multiplex){
217             case 0: LATCbits.LATC7 = 1; break;
218             case 1: LATCbits.LATC6 = 1; break;
219             case 2: LATCbits.LATC1 = 1; break;
220             case 3: LATCbits.LATC0 = 1; break;
221         }
222         multiplex++;
223         if(multiplex > 3) multiplex = 0;
224         cntISR++; // +1 c/ 2.5ms
225
226         INTCONbits.TMROIF = 0;
227     }
228 }
229
230 // =====

```

```

231 //                                     MAIN
232 // =====
233 void main(void){
234     unsigned char MedioSecCount = 0; // 0 o 1, c/ 500ms: 0.5seg
235     unsigned char cuenta_Minutos = 0;
236
237     int btn_A_cnt = 0; // Botones col 4 Teclado
238     int btn_B_cnt = 0;
239     int btn_C_cnt = 0;
240     int btn_D_cnt = 0;
241     char ing_btn_A = 0; // Presionar A para usar reloj
242
243     // --- CONFIGURACION CRITICA PARA USAR RC4 y RC5 ---
244     UCONbits USBEN = 0; // Deshabilita el modulo USB
245     UCFGbits.UTRDIS = 1; // Deshabilita el transceptor USB (E/S digital)
246     ADCON1 = 0x0F; // Buzzer en RAO
247     // -----
248
249     // --- Configuración de Puertos ---
250     TRISD = 0;
251     TRISCbits.TRISC0 = 0;
252     TRISCbits.TRISC1 = 0;
253     TRISCbits.TRISC6 = 0;
254     TRISCbits.TRISC7 = 0;
255     // TRISC |= 0b00110000; // Activar s2 y s3
256
257     // ===== Port B - Teclado 4*4 Matricial =====
258     // RB0-RB3: Entradas (Columnas), RB4-RB7: Salidas (Filas)
259     TRISB = 0b00001111; // // 0xFF: Nada presionado
260     LATB = 0b11110000; // // filas en 1
261     // INTCON2bits.RBPU = 0; // Resist. Pull-Up internas PIC(NO fisicas)
262
263     LATD = 0;
264     LATCbits.LATC7 = 0;
265     LATCbits.LATC6 = 0;
266     LATCbits.LATC1 = 0;
267     LATCbits.LATC0 = 0;
268
269     // Puerto A para Buzzer
270     TRISAbits.TRISA0 = 0;
271     LATAbits.LATA0 = 0;
272
273     // ==== INTERRUPTACIONES DE 2.5 ms ====
274     // Tiempo total del ciclo = 4 displays * 2.5 ms = 10 ms
275     // =====
276     // TMR0: 5ms (Fosc=48MHz, 16-bit, Prescaler 1:8)
277     // =====
278     // Tcy = 1 / (48MHz / 4) = 83.33ns
279     // Tick = Tcy * 8 (Prescaler) = 666.67ns
280     // Conteos = 2.5ms / 666.67ns = 3750
281     // Carga = 65536 - 3750 = 61786 = 0xF15A
282     TMR0H = 0xF1;
283     TMR0L = 0x5A;
284     TOCON = 0b00000010;
285
286     // Configuración del TOCON:
287     // bit 7: TMR0ON = 0 (Apagado por ahora)
288     // bit 6: T08BIT = 0 (Configurado en 16 bits)
289     // bit 5: T0CS = 0 (Reloj interno Fosc/4)
290     // bit 4: T0SE = 0 (No reloj interno)
291     // bit 3: PSA = 0 (Prescaler activado)
292     // bit 2-0: T0PS = 010 (Prescaler 1:8)

```

```

293 RCONbits.IPEN      = 1;
294 INTCON2bits.TMROIP = 1;
295 INTCONbits.TMROIE  = 1;
296 INTCONbits.TMROIF  = 0;
297 INTCONbits.GIEH    = 1;
298 INTCONbits.GIEL    = 1;
299 TOCONbits.TMROON   = 1;
300
301
302 // INICIO - LED RDO *****
303 for(int i = 0; i < 10; i++){
304     LATDbits.LATDO = !LATDbits.LATDO;
305     // delay bloqueante: L = entero largo (desborde)
306     for(long d = 0; d < 43000L; d++); // ~50ms a 48MHz
307 }
308 LATDbits.LATDO = 1;
309
310 ActualizarDisplays(); //0000
311 char tecla;
312
313 while(1){
314     // ---- Tick de 500ms : (500/2.5 = 200 int.) ----
315     if(cntISR >= 200){
316         cntISR -= 200;    // -200 count ISR para los seg
317
318         MedioSecCount = !MedioSecCount;
319         LATDbits.LATDO = !LATDbits.LATDO; // LED 1Hz
320
321         if(!reloj_activo){
322             if(modo_display){
323                 ActualizarDisplays(); // Muestra rEL
324             } else {
325                 // PARPADEO:
326                 if(MedioSecCount == 0){
327                     datos[0] = 0x00; // OFF Displays
328                     datos[1] = 0x00;
329                     datos[2] = 0x00;
330                     datos[3] = 0x00;
331                 } else {
332                     ActualizarDisplays(); // HH:MM
333                 }
334             }
335         } else{
336             if(MedioSecCount == 0) cuenta_Minutos++; // 60 SEG = +1 MIN
337             if(cuenta_Minutos >= 60){
338                 cuenta_Minutos = 0;
339                 Minutos++;
340                 if(Minutos > 59){
341                     Minutos = 0; Hora++;
342                 }
343                 if(Hora > 23) Hora = 0; // 24 horas (0 a 23)
344             }
345             ActualizarDisplays();
346         }
347     }
348
349     tecla = LeerTeclado();
350     // ---- Tecla A: rEL * 2 seg + 0000
351     if(tecla == 10 && !reloj_activo){
352         btn_A_cnt++; // Antirebote
353         if(btn_A_cnt == 1){
354             Buzzer();
355             modo_display = 1; // rEL

```

```

356         ActualizarDisplays();
357         DelayXms(2000); // rEL * 2 seg
358         modo_display = 0; Hora = 0; Minutos = 0; //0000
359         ActualizarDisplays();
360         ing_btn_A = 1;
361     }
362 } else {
363     btn_A_cnt = 0; // Setear Antirebote
364 }
365
366 // ---- Tecla B: Incrementar Tiempo ----
367 if(tecla == 11 && !reloj_activo && ing_btn_A){
368     btn_B_cnt++; // Antirebote
369     if(btn_B_cnt == 1){
370         Buzzer();
371         IncrementarTiempo();
372     }
373     if(btn_B_cnt > 20 && (btn_B_cnt % 3) == 0){
374         IncrementarTiempo();
375     }
376 } else {
377     btn_B_cnt = 0; // set Antirebote
378 }
379
380 /* -----
381  * t_esp=20 y velo=3 Para inc./dec. al pulsar
382  * 1 while = (3 filas teclado * 5ms) + 50ms delay + 5ms = 70ms.
383  * * 20 vueltas * 70ms = 1400ms = 1.4 seg. // modo presionado auto.
384  * * % 3 : (3 vueltas * 70ms = 210ms).
385  * velocidad cambia el tiempo cada 210ms.
386  * -----
387 */
388
389 // ---- Tecla C: Decrementar Tiempo ----
390 if(tecla == 12 && !reloj_activo && ing_btn_A){
391     btn_C_cnt++; // Antirebote
392     if(btn_C_cnt == 1){
393         Buzzer();
394         DecrementarTiempo();
395     }
396     if(btn_C_cnt > 20 && (btn_C_cnt % 3) == 0){
397         DecrementarTiempo();
398     }
399 } else {
400     btn_C_cnt = 0; // set Antirebote
401 }
402
403 // ---- Tecla D: Fijar / Setear Hora ----
404 if(tecla == 13 && !reloj_activo && ing_btn_A){
405     btn_D_cnt++;
406     if(btn_D_cnt == 1){
407         Buzzer(); reloj_activo = 1; // reloj
408         cuenta_Minutos = 0; cntISR = 0;
409         ActualizarDisplays();
410     }
411 } else {
412     btn_D_cnt = 0;
413 }
414 DelayXms(5);
415 }
416 return;
417 }

```


Listing 19: Pr4_Parte2.c – Reloj Digital

```

1  /*
2  * File:    Pr4_Parte3.c
3  * Author:  JAIME CHIQUI
4  * Created on 4 de junio de 2026, 20:29
5  */
6
7  //  PRACTICA 3 - PARTE 3: Operacion Microondas
8
9  #include <xc.h>
10
11  // =====
12  //                      VARIABLES GLOBALES
13  // =====
14  volatile char datos[4] = {0x00, 0x00, 0x00, 0x00}; //bits ON los segmentos
15  volatile char multiplex = 0;
16  volatile char digitos[4] = {0, 0, 0, 0};    // Numero a ON display
17
18  // Flags, HH:MM y contadores de ISR
19  volatile unsigned int cntISR = 0; // contador de interrupciones TMR0
20  unsigned char Hora = 0;
21  unsigned char Minutos = 0;
22  char reloj_activo = 1; // 0 = ajuste (parpadea), 1 = fija (reloj)
23  char modo_display = 0; // 1 = rEL
24  unsigned char cuenta_Minutos = 0;
25
26  // Operaci n Microondas
27  unsigned char Seg_Micro = 0; // Seg regresivo
28  unsigned char Min_Micro = 0; // Min regresivo
29  char op_Micro = 0; // 0 = Reposo/Ajuste, 1 = ON
30  char ing_micro = 0; // 1 ingresando tiempo
31  char reloj_no_configurado = 1;
32
33  // Control del Bloqueo para Niños y Puerta (Interlock)
34  unsigned char bloqueo_ninos = 0; // 0 = desbloqueado, 1 = bloqueado
35  unsigned char puerta_abierta = 0; // 0 = close, 1 = open puerta: #
36  unsigned int timer_stop_3s = 0; // Cont. 3 seg
37  unsigned char conteo_paradas = 0; // Cont. pulsaciones tecla *
38  // 1 *: Pausa, 2*: Clear time, 3seg *: Bloquear y 3seg*: desbloquear
39
40  //Codigo de los numeros 0-9 (c todo com n)
41  const char codigosNum[12] = {
42      0b11111100, // 0
43      0b01100000, // 1
44      0b11011010, // 2
45      0b11110010, // 3
46      0b01100110, // 4
47      0b10110110, // 5
48      0b10111110, // 6
49      0b11100000, // 7
50      0b11111110, // 8
51      0b11110110, // 9
52      0b00101010, // n
53      0b10001110 // F
54  };
55  //Mostrar rEL
56  const char rEL[4] = {
57      0b00001010, // D1 'r'
58      0b10011110, // D2 'E'
59      0b00011100, // D3 'L'
60      0b00000000 // D4 OFF

```

```

61 };
62 // =====
63 //             CONFIGURACION DE INTERRUPTACIONES
64 // =====
65 void ConfiguracionInterrupciones_TMRO(void) {
66     RCONbits.IPEN = 1; // Prioridades de interrupci n
67     INTCON2bits.TMROIP = 1; // Timer0 con HP
68     INTCONbits.TMROIE = 1; // Interrupci n Timer0
69     INTCONbits.TMROIF = 0; // Limpiar bandera del Timer0
70     INTCONbits.GIEH = 1; // Interrupciones globales de HP
71     INTCONbits.GIEL = 1; // Interrupciones globales de LP
72
73     TOCONbits.TMROON = 1; // ON/OFF Timer0
74 }
75 // =====
76 //             SETEAR VALORES (0)
77 // =====
78 // Pause Microondas
79 void DetenerMicroondas(void){
80     op_Micro = 0;
81     ing_micro = 0;
82     LATDbits.LATD0 = 0; // Apaga de inmediato el magnetron
83 }
84 // ==== Clear Microondas
85 void LimpiarMicroondas(void){
86     DetenerMicroondas();
87
88     Min_Micro = 0;
89     Seg_Micro = 0;
90     for(char i = 0; i < 4; i++) digitos[i] = 0;
91 }
92 void ReiniciarReloj(void) {
93     Hora = 0;
94     Minutos = 0;
95     cntISR = 0;
96     cuenta_Minutos = 0;
97     reloj_activo = 0;
98     reloj_no_configurado = 1;
99 }
100 // ajustar Hora desde donde esta conf.
101 void AjustarReloj(void){
102     reloj_activo = 0;
103     cntISR = 0;
104     cuenta_Minutos = 0;
105 }
106
107 // =====
108 //             DELAYS
109 // =====
110 void Delay1ms() {
111     int veces;
112     for (veces = 0; veces <= 860; veces++);
113 }
114
115 void DelayXms(int Milis) {
116     for (int i = 0; i <= Milis; i++) {
117         Delay1ms();
118     }
119 }
120 void ApagarDisplays(void)
121 {
122     for(char i=0;i<4;i++) datos[i]=0;
123 }

```

```

124 // =====
125 //          ACTUALIZAR DISPLAYS + TECLA INGRESADA
126 // =====
127 void AsignarTiempoDisplay(unsigned char valor_izq, unsigned char valor_der) {
128     // Valor 2310: izq=23 derecha=10
129     // ejemplo: 37 / 10 = 3.7 (int en c: 3)
130     //          37 % 10 = 7
131     datos[0] = codigosNum[valor_izq / 10]; // Decena izq (HH o Min Micro)
132     datos[1] = codigosNum[valor_izq % 10]; // Unidad izq (HH o Min Micro)
133     datos[2] = codigosNum[valor_der / 10]; // Decena derecha (Min o Seg Micro)
134     datos[3] = codigosNum[valor_der % 10]; // Unidad derecha (Min o Seg Micro)
135 }
136
137 void ActualizarDisplays(void) {
138     if (bloqueo_ninos) { // Si est bloqueado, dibuja "----"
139         datos[0] = 0b10011100; // D1: Forma '['
140         datos[1] = 0b10010000; // D2: segmentos a, d
141         datos[2] = 0b10010000; // D3: segmentos a, d
142         datos[3] = 0b11110000; // D4: Forma ']'
143     } else if (modo_display) { //Mostrar palabra rEL
144         datos[0] = rEL[0]; // D1: r
145         datos[1] = rEL[1]; // D2: E
146         datos[2] = rEL[2]; // D3: L
147         datos[3] = rEL[3]; // D4: OFF
148     } else if (op_Micro || ing_micro || (Min_Micro > 0 || Seg_Micro > 0)) {
149         // Env a los Min y Seg del microondas a los displays
150         AsignarTiempoDisplay(Min_Micro, Seg_Micro);
151     } else {
152         // Env a la Hora y los Min del reloj a los displays
153         AsignarTiempoDisplay(Hora, Minutos);
154     }
155     // Aviso de puerta abierta
156     if (puerta_abierta) {
157         datos[0] = 0b10011100; // [
158     }
159 }
160 // =====
161 //          DESPLAZAR VALORES A LA IZQUIERDA + TECLA INGRESADA
162 // =====
163 void Desp_izquierda(char tecla_ingresada) {
164     digitos[0] = digitos[1]; // D1
165     digitos[1] = digitos[2]; // D2
166     digitos[2] = digitos[3]; // D3
167     digitos[3] = tecla_ingresada; //D4
168 }
169
170 void IncrementarTiempo(void) {
171     Minutos++;
172     if (Minutos > 59) {
173         Minutos = 0;
174         Hora++;
175     }
176     if (Hora > 23) Hora = 0; // Horas
177     ActualizarDisplays();
178 }
179
180 void DecrementarTiempo(void) {
181     if (Minutos == 0) {
182         Minutos = 59;
183         if (Hora == 0) Hora = 23; // Horas
184         else Hora--;
185     } else {
186         Minutos--;

```

```

187     }
188     ActualizarDisplays();
189 }
190 // =====
191 //          BUZZER
192 // =====
193 void Buzzer(void) { // Buzzer pin 2 pic
194     LATAbits.LATA0 = 1; // ON buzzer
195     DelayXms(100); // Delay 100ms
196     LATAbits.LATA0 = 0; // OFF buzzer
197 }
198 // =====
199 //          LEER TECLADO MATRICIAL (Filas RB4-RB7, Col RBO-RB3)
200 // =====
201 char LeerTeclado(void) {
202     char tecla = 0xFF; // 0xFF: Nada presionado
203
204     // -----
205     LATB = 0b01110000; // Fila 1
206     DelayXms(5); // Tiempo espera
207     switch (PORTB) {
208         // f1_input: 0111 f1_out: xxxx
209         case 0b01110111: tecla = 1; break;
210         case 0b01111011: tecla = 2; break;
211         case 0b01111101: tecla = 3; break;
212         case 0b01111110: tecla = 10; break; // Tecla 'A'
213     }
214
215     // -----
216     LATB = 0b10110000; // Fila 2:
217     DelayXms(5);
218     switch (PORTB) {
219         // f2_input: 1011 f2_out: xxxx
220         case 0b10110111: tecla = 4; break;
221         case 0b10111011: tecla = 5; break;
222         case 0b10111101: tecla = 6; break;
223         case 0b10111110: tecla = 11; break; // Tecla 'B'
224     }
225     // -----
226     LATB = 0b11010000; // Fila 3
227     DelayXms(5);
228     switch (PORTB) {
229         // f3_input: 1101 f3_out: xxxx
230         case 0b11010111: tecla = 7; break;
231         case 0b11011011: tecla = 8; break;
232         case 0b11011101: tecla = 9; break;
233         case 0b11011110: tecla = 12; break; // Tecla 'C'
234     }
235     // -----
236     LATB = 0b11100000; // Fila 4:
237     DelayXms(5);
238     switch (PORTB) {
239         // f4_input: 1110 f4_out: xxxx
240         case 0b11100111: tecla = 14; break; // Tecla '*'
241         case 0b11101011: tecla = 0; break; // 0
242         case 0b11101101: tecla = 15; break; // Tecla '#'
243         case 0b11101110: tecla = 13; break; // Tecla 'D'
244     }
245     // Anti-rebote de 50ms
246     if (tecla != 0xFF) { // presiona algo
247         DelayXms(50);
248         return tecla;
249     }

```

```

250     return tecla; //tecla ingresada
251 }
252 // =====
253 //             ISR ALTA PRIORIDAD - TMR0 16 bits
254 // =====
255 void __interrupt(high_priority) InterrupcionHP(void) {
256     if (INTCONbits.TMR0IF) {
257         // ==== INTERRUPTACIONES DE 2.5 ms ====
258         // Tiempo total del ciclo = 4 displays * 2.5 ms = 10 ms
259         // =====
260         // TMR0: 2.5ms (Fosc=48MHz, 16-bit, Prescaler 1:8)
261         // =====
262         // Tcy = 1 / (48MHz / 4) = 83.33ns
263         // Tick = Tcy * 8 (Prescaler) = 666.67ns
264         // Conteos = 2.5ms / 666.67ns = 3750
265         // Carga = 65536 - 3750 = $61786 = 0xF15A
266         TMR0H = 0xF1;
267         TMR0L = 0x5A;
268
269         // OFF DISPLAY(evita ghosting)
270         LATCbits.LATC7 = 0; // D1
271         LATCbits.LATC6 = 0; // D2
272         LATCbits.LATC1 = 0; // D3
273         LATCbits.LATC0 = 0; // D4
274
275         // Multiplexaci n
276         LATD = (datos[multiplex] & 0xFE) | (LATDbits.LATD0);
277         switch (multiplex) {
278             case 0: LATCbits.LATC7 = 1;
279                     break;
280             case 1: LATCbits.LATC6 = 1;
281                     break;
282             case 2: LATCbits.LATC1 = 1;
283                     break;
284             case 3: LATCbits.LATC0 = 1;
285                     break;
286         }
287         multiplex++;
288         if (multiplex > 3) multiplex = 0;
289         cntISR++; // +1 c/ 2.5ms
290
291         INTCONbits.TMR0IF = 0;
292     }
293 }
294
295 // =====
296 //             MAIN
297 // =====
298 void main(void) {
299     unsigned char MedioSecCount = 0; // 0 o 1, c/ 500ms: 0.5seg
300     unsigned char btn_A_cnt = 0; // Botones col 4 Teclado
301     unsigned char btn_B_cnt = 0;
302     unsigned char btn_C_cnt = 0;
303     unsigned char btn_D_cnt = 0;
304
305     // --- CONFIGURACI N CR TICA PARA USAR RC4 y RC5 ---
306     UCONbits.USBEN = 0; // Deshabilita el m dulo USB
307     UCFGbits.UTRDIS = 1; // Deshabilita el transceptor USB (E/S digital)
308     ADCON1 = 0x0F; // Buzzer en RA0
309     // -----
310
311     // --- Configuraci n de Puertos ---
312     TRISD = 0;

```

```

313 TRISCBits.TRISCO = 0;
314 TRISCBits.TRISC1 = 0;
315 TRISCBits.TRISC6 = 0;
316 TRISCBits.TRISC7 = 0;
317 // TRISC |= 0b00110000; // Activar s2 y s3
318
319 // ===== Port B - Teclado 4*4 Matricial =====
320 TRISB = 0b00001111; // 0xFF: Nada presionado
321 LATB = 0b11110000; // filas en 1
322 // INTCON2bits.RBPV = 0; // Resist. Pull-Up internas PIC(NO fisicas)
323
324 LATD = 0;
325 LATCBits.LATC7 = 0;
326 LATCBits.LATC6 = 0;
327 LATCBits.LATC1 = 0;
328 LATCBits.LATC0 = 0;
329
330 // Puerto A para Buzzer
331 TRISAbits.TRISA0 = 0;
332 LATAbits.LATA0 = 0;
333
334 // ==== INTERRUPTACIONES DE 2.5 ms ====
335 // Carga = 65536 - 3750 = 61786 = 0xF15A
336 TMR0H = 0xF1;
337 TMR0L = 0x5A; // calculos InterrupcionHp
338 TOCON = 0b00000010;
339
340 // Configuraci n del TOCON:
341 // bit 7: TMR0ON = 0 (Apagado por ahora)
342 // bit 6: T08BIT = 0 (Configurado en 16 bits)
343 // bit 5: T0CS = 0 (Reloj interno Fosc/4)
344 // bit 4: T0SE = 0 (No reloj interno)
345 // bit 3: PSA = 0 (Prescaler activado)
346 // bit 2-0: TOPS = 010 (Prescaler 1:8)
347
348 // =====
349 ConfiguracionInterrupciones_TMRO();
350
351 // INICIO - LED RDO *****
352 for (int i = 0; i < 10; i++) {
353     LATDBits.LATD0 = !LATDBits.LATD0;
354     // delay bloqueante: L = entero largo (desborde)
355     for (long d = 0; d < 43000L; d++); // ~50ms a 48MHz
356 }
357 LATDBits.LATD0 = 1;
358
359 ActualizarDisplays(); //0000
360 char tecla;
361
362 while (1) {
363     // ---- Tick de 500ms : (500/2.5 = 200 int.) ----
364     if (cntISR >= 200) {
365         cntISR -= 200; // -200 count ISR para los seg
366         MedioSecCount = !MedioSecCount;
367
368         if (!op_Micro) {
369             LATDBits.LATD0 = 0; // LED 1Hz en reposo
370         }
371
372         if (!reloj_activo) {
373             if (modo_display) {
374                 ActualizarDisplays(); // Muestra rEL

```

```

375         } else {
376             // PARPADEO:
377             if (MedioSecCount == 0) {
378                 ApagarDisplays();
379             } else {
380                 ActualizarDisplays(); // HH:MM
381             }
382         }
383     } else {
384         //microondas NO funcionando, Reloj HHMM
385         if (MedioSecCount == 0) cuenta_Minutos++; // 60 SEG = +1 MIN
386         if (cuenta_Minutos >= 60) {
387             cuenta_Minutos = 0;
388             Minutos++;
389             if (Minutos > 59) {
390                 Minutos = 0;
391                 Hora++;
392             }
393             if (Hora > 23) Hora = 0; // 24 horas (0 a 23)
394         }
395         // Decrementar contador del microondas cada 1 seg
396         if (op_Micro && MedioSecCount == 0) {
397             if (puerta_abierta) {
398                 // SEGURIDAD
399                 DetenerMicroondas();
400             }
401             else if (Min_Micro == 0 && Seg_Micro == 0) {
402                 // CASO 1: Lleg a 00:00 -> Terminar operaci n
403                 LimpiarMicroondas();
404                 reloj_activo = 1;
405
406                 for(unsigned char b = 0; b < 3; b++) {
407                     Buzzer(); //
408
409                     DelayXms(1000);
410                 }
411             }
412             else if (Seg_Micro == 0) {
413                 // CASO 2: Quedan min el seg esta 0 -> -1 min
414                 Min_Micro--;
415                 Seg_Micro = 59;
416             }
417             else {
418                 Seg_Micro--; // CASO 3: -1 seg
419             }
420         }
421         // ---- Control Pantalla ----
422         if (reloj_no_configurado && !op_Micro && !ing_micro &&
MedioSecCount == 0) {
423             ApagarDisplays();
424         } else {
425             ActualizarDisplays();
426         }
427     }
428
429     tecla = LeerTeclado();
430     // Bloquear todas las teclas mod ni os
431     if (bloqueo_ninos && tecla != 14) tecla = 0xFF;
432
433     // ---- Tecla A: rEL * 2 seg + 0000
434     if (tecla == 10 && !op_Micro) {
435         btn_A_cnt++; // Antirebote

```

```

436         if (btn_A_cnt == 1) {
437             Buzzer();
438             modo_display = 1; // rEL
439             ActualizarDisplays();
440             DelayXms(2000); // rEL * 2 seg
441             modo_display = 0;
442             LimpiarMicroondas();
443             if(reloj_no_configurado){
444                 ReiniciarReloj(); // 1era conf. -> 00:00
445             }
446             else{
447                 AjustarReloj(); // conserva HHMM
448             }
449
450             ActualizarDisplays();
451         }
452     } else {
453         btn_A_cnt = 0; // Setear Antirebote
454     }
455
456     // ---- Teclas 0-9: En estado reposo ----
457     if (tecla >= 0 && tecla <= 9 && reloj_activo && !op_Micro) {
458         // Desplazamiento num.
459         Desp_izquierda(tecla);
460         ing_micro = 1;
461
462         // Se pasa a minutos y segundos
463         Min_Micro = (digitos[0] * 10) + digitos[1];
464         Seg_Micro = (digitos[2] * 10) + digitos[3];
465
466         Buzzer();
467         ActualizarDisplays();
468         DelayXms(250);
469     }
470
471     // ---- Tecla B: Incrementar Tiempo ----
472     if (tecla == 11 && !reloj_activo) {
473         btn_B_cnt++; // Antirebote
474         if (btn_B_cnt == 1) {
475             Buzzer();
476             IncrementarTiempo();
477         }
478         if (btn_B_cnt > 20 && (btn_B_cnt % 3) == 0) {
479             IncrementarTiempo();
480         }
481     } else {
482         btn_B_cnt = 0; // set Antirebote
483     }
484
485     /* -----
486     * t_espera: tiempo a tener presionado, velo: rapidez del valor a inc/
487     dec
488     * t_esp=20 y velo=3 Para inc./dec. al pulsar
489     * 1 while = (3 filas teclado * 5ms) + 50ms delay + 5ms = 70ms.
490     * * 20 vueltas * 70ms = 1400ms = 1.4 seg. // modo presionado auto.
491     * * % 3 : (3 vueltas * 70ms = 210ms).
492     * velocidad cambia el tiempo cada 210ms.
493     * -----
494     */
495
496     // ---- Tecla C: Decrementar Tiempo ----
497     if (tecla == 12 && !reloj_activo) {
498         btn_C_cnt++; // Antirebote

```



```

497         if (btn_C_cnt == 1) {
498             Buzzer();
499             DecrementarTiempo();
500         }
501         if (btn_C_cnt > 20 && (btn_C_cnt % 3) == 0) {
502             DecrementarTiempo();
503         }
504     } else {
505         btn_C_cnt = 0; // set Antirebote
506     }
507
508     // ---- Tecla D: Fijar / +30 seg ----
509     if (tecla == 13) {
510         btn_D_cnt++;
511         if (btn_D_cnt == 1) {
512             Buzzer();
513             // Caso A: modo ajuste, fija la hora
514             if (!reloj_activo && !op_Micro) {
515                 reloj_activo = 1;
516                 cuenta_Minutos = 0;
517                 cntISR = 0;
518                 reloj_no_configurado = 0;
519             } // Caso B: Reposo (Reloj) + D, inicia con 30s
520             else if (reloj_activo && !op_Micro) {
521                 op_Micro = 1;
522                 ing_micro = 0;
523                 if (Min_Micro == 0 && Seg_Micro == 0) {
524                     Seg_Micro = 30; //0030
525                 }
526                 LATDbits.LATD0 = 1; // LED RDO ON (motor y magnetron)
527             } // Caso C: microondas FUNCIONANDO, +30S
528             else if (op_Micro) {
529                 Seg_Micro += 30;
530                 if (Seg_Micro >= 60) {
531                     Seg_Micro -= 60;
532                     Min_Micro++;
533                 }
534             }
535             ActualizarDisplays();
536         }
537     } else {
538         btn_D_cnt = 0;
539     }
540
541     // ---- Tecla *: STOP / CLEAR / BLOQUEO DE NI OS ----
542     if (tecla == 14) {
543         timer_stop_3s++; // Incrementa presionada
544
545         // Cada ciclo con delays del teclado 70ms.
546         // 3000ms / 70ms = 43 ciclos para 3 seg.
547         if (timer_stop_3s >= 43) {
548             timer_stop_3s = 0; // Resetea contador
549
550             if (!op_Micro && !ing_micro) {
551                 // Si se desactiva el bloqueo, limpiamos los datos viejos
552                 bloqueo_ninos = !bloqueo_ninos;
553                 if (!bloqueo_ninos) {
554                     LimpiarMicroondas();
555                 }
556
557                 ActualizarDisplays();
558                 LATAbits.LATA0 = 1;
559                 DelayXms(300);

```

```

560         LATAbits.LATA0 = 0;
561
562         conteo_paradas = 0;
563         DelayXms(5); // Evita ghosting
564     }
565 }
566
567 // Pausa / Borrar al soltar o presionar por primera vez
568 if (timer_stop_3s == 1 && !bloqueo_ninos) {
569     Buzzer();
570     if (op_Micro) { // 1 Pulsaci n cocinando: PAUSA
571         DetenerMicroondas();
572         conteo_paradas = 1;
573     } else {
574         // Pulsaci n en reposo o 2da pulsaci n: CLEAR
575         LimpiarMicroondas();
576         conteo_paradas = 0;
577     }
578     ActualizarDisplays();
579 }
580 } else { // Si se suelta la tecla *
581     timer_stop_3s = 0;
582 }
583
584 // ---- Tecla #: PUERTA ABIERTA/CERRADA ----
585 if (tecla == 15 && !bloqueo_ninos) {
586     Buzzer();
587     puerta_abierta = !puerta_abierta; // Invierte el estado de la
puerta
588
589     if (puerta_abierta) {
590         // Si la puerta se abre, fuerza el apagado
591         DetenerMicroondas();
592         Buzzer(); DelayXms(100); Buzzer();
593     }
594
595     ActualizarDisplays();
596     DelayXms(300); // Delay de antirrebote
597 }
598
599     DelayXms(5);
600 }
601 return;
602 }

```

Listing 20: Pr4_Parte4.c – Modo Ninos (Bloqueo). Parte 3 y 4

9. Preguntas

1. ¿Cómo se generalizaría el procedimiento realizado en esta práctica para leer un teclado de 10×8 , es decir, de 80 teclas?

Respuesta:

El mismo principio de multiplexación matricial se extiende directamente:

- a) **Configuración de puertos:** Se requieren 10 pines como salidas (filas) y 8 pines como entradas (columnas) con resistencias de pull-up (internas o físicas), totalizando 18 pines, frente a los 80 que necesitaría una conexión directa.
- b) **Estructura de escaneo con bucles anidados:**

```
1 char LeerTeclado10x8(void) {
2     for(char fila = 0; fila < 10; fila++) {
3         ActivarFila(fila);    // 0 fila activa
4         DelayXms(5);          // Estabilizacion
5         for(char col = 0; col < 8; col++) {
6             if(LeerColumna(col) == 0) {
7                 DelayXms(50);    // Antirebote
8                 return fila * 8 + col; //Codigo 0-79
9             }
10        }
11    }
12    return 0xFF; // Ninguna tecla presionada
13 }
```

Listing 21: Generalización para teclado 10x8

- c) **Maapeo de teclas:** Cada tecla recibe un identificador único según Código = Fila $\times$ 8 + Columna, con rango de 0 a 79.
- d) **Tiempo de escaneo completo:**
Usando el mismo código utilizado en la práctica, cada fila requeriría únicamente el tiempo de estabilización (5 ms), el delay de 50 ms se ejecuta solamente cuando se detecta una tecla:

```
1 if(tecla != 0xFF){
2     DelayXms(50);
3     return tecla;
4 }
```

Por lo que un barrido completo tomaría:

$$10 \times 5 \text{ ms} = 50 \text{ ms}$$

El retardo adicional de 50 ms se aplica únicamente cuando se detecta una pulsación para realizar el antirebote.

2. En el circuito de la Figura 1.1: ¿cuál es la función de las 4 resistencias de 10k ubicadas en los terminales de las conexiones de las columnas del teclado matricial hacia VCC?

Respuesta:

Son resistencias de **pull-up** y cumplen tres funciones esenciales:

- **Definir nivel lógico en reposo:** Cuando ninguna tecla está presionada, las entradas de las columnas no reciben un nivel lógico definido desde el teclado. Las resistencias pull-up fuerzan dichas líneas a nivel alto +5V, evitando estados flotantes, garantizando que el PIC lea un nivel alto estable.
- **Permitir la detección de presión:** Al activar una fila a 0V y presionar una tecla, la columna correspondiente es llevada a tierra a través del contacto. La transición de alto a bajo es detectada limpiamente por el microcontrolador.

- **Limitar la corriente de cortocircuito:**

$$I_{\text{máx}} = \frac{5 \text{ V}}{10 \text{ k}\Omega} = 0,5 \text{ mA}$$

Este valor es seguro para los pines del PIC18F4550 y evita daños en el microcontrolador cuando fila y columna se conectan simultáneamente.

3. ¿Se podría eliminar del esquema las 4 resistencias mencionadas en la pregunta anterior? Explique considerando que se utiliza un PIC18F4550 y que el teclado se conecta en el Puerto B.

Respuesta:

Sí, podrían eliminarse físicamente, pero únicamente si se habilitan las resistencias pull-up internas del Puerto B del PIC18F4550. Este microcontrolador incorpora pull-ups internos configurables en RB0:RB7 a través del bit RBPU del registro INTCON2:

```
1 INTCON2bits.RBPU = 0; // 0 = pull-ups internos habilitados
```

Listing 22: Habilitación de pull-ups internos en Puerto B

Sin embargo, existen consideraciones importantes:

- Los pull-ups internos del Puerto B presentan una resistencia equivalente elevada. De acuerdo con la hoja de datos del PIC18F4550, la corriente de los weak pull-ups del PORTB (IPURB) puede variar entre $50 \mu\text{A}$ y $400 \mu\text{A}$ para $V_{DD} = 5 \text{ V}$.

	IPU	Weak Pull-up Current				
D070	IPURB	PORTB Weak Pull-up Current	50	400	μA	$V_{DD} = 5 \text{ V}, V_{PIN} = V_{SS}$
D071	IPURD	PORTD Weak Pull-up Current	50	400	μA	$V_{DD} = 5 \text{ V}, V_{PIN} = V_{SS}$

Figura 8: DC Characteristics: PIC18F2455/2550/4455/4550

Esto corresponde aproximadamente a una resistencia equivalente entre:

$$R = \frac{V}{I}$$

$$R_{\text{min}} = \frac{5 \text{ V}}{400 \mu\text{A}} = 12,5 \text{ k}\Omega$$

$$R_{\text{max}} = \frac{5 \text{ V}}{50 \mu\text{A}} = 100 \text{ k}\Omega$$

Por esta razón, los pull-ups internos proporcionan una corriente menor que las resistencias externas de $10 \text{ k}\Omega$ y pueden resultar más sensibles al ruido en aplicaciones con cables largos o ambientes con interferencia.

- Si los pull-ups internos **no** se habilitan y las resistencias externas se eliminan, los pines RB0:RB3 quedarán flotantes, causando lecturas erróneas e impredecibles. En ese caso las resistencias externas son **obligatorias**.

En la implementación realizada las resistencias externas de 10 k son indispensables, en el programa desarrollado las resistencias pull-up internas no fueron habilitadas, ya que la línea correspondiente quedó comentada.

4. ¿Cuál es el objetivo de colocar una demora de 50 ms entre cada bloque de monitoreo de las filas en la subrutina de lectura del teclado matricial?

Respuesta:

El delay de 50 ms cumple la función de **antirebote**:

- a) Los contactos mecánicos de los botones no cierran limpiamente. En los primeros milisegundos de presión, el contacto abre y cierra decenas de veces antes de estabilizarse (rebote), generando múltiples transiciones lógicas.
- b) **Efecto sin antirebote:** Sin el delay, el microcontrolador detectaría la misma pulsación física como múltiples presiones consecutivas, corrompiendo el valor ingresado en los displays.
- c) **Por qué 50 ms:** Los rebotes mecánicos de botones de membrana como los del teclado matricial se estabilizan típicamente en 10–30 ms. Usar 50 ms añade margen de seguridad suficiente para cualquier variabilidad del componente.
- d) **Complemento del delay de 5 ms:** El delay de 5 ms aplicado tras activar cada fila cumple una función diferente: permite que la línea alcance niveles lógicos estables antes de leer las columnas, evitando lecturas transitorias durante la conmutación.

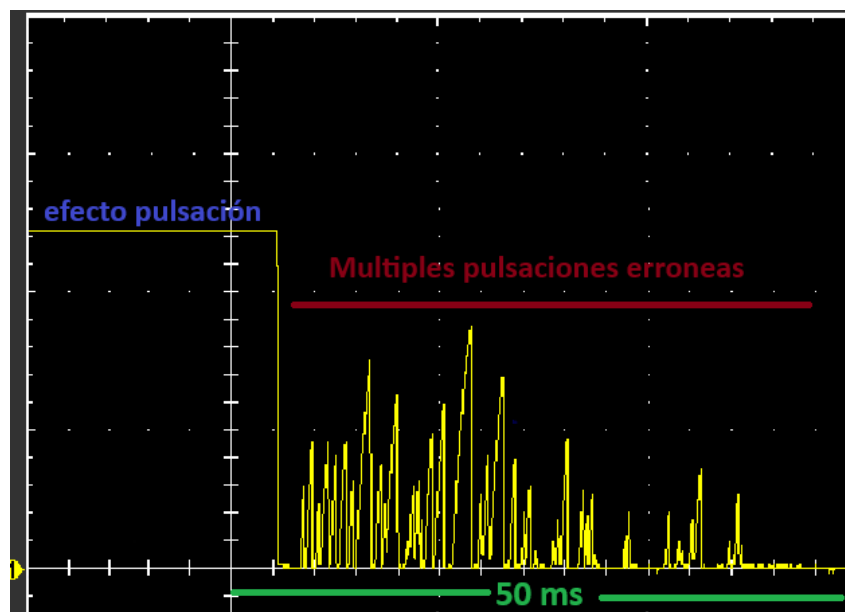


Figura 9: Antirrebote de un pulsador de membrana.

5. Dibuje el diagrama del circuito para manejar el buzzer asociado al pulsado de las teclas del teclado matricial.

Respuesta:

El buzzer se controla mediante un transistor NPN 2N2222 conectado al pin RA0 del PIC18F4550, como se muestra en el diagrama de la Figura 4

Principio de operación:

- **RA0 = 1 (alto):** Transistor en saturación → colector-emisor en cortocircuito → buzzer energizado (**suena**).
- **RA0 = 0 (bajo):** Transistor en corte → circuito abierto → buzzer sin corriente (**silencio**).
- La resistencia 1 kΩ limita la corriente de base a $I_B = \frac{5-0,7}{1\text{k}\Omega} = 4,3\text{mA}$, garantizando saturación completa y protegiendo el pin RA0 del PIC.
- El diodo Zener (opcional) conectado en paralelo con el buzzer actúa como elemento de supresión de sobretensiones. Durante el apagado del transistor, cualquier pico de tensión generado por la carga es limitado por la tensión de ruptura del Zener, evitando que aparezcan diferencias de potencial elevadas entre colector y emisor del transistor NPN. Esta protección reduce el riesgo de daño en el 2N2222 y mejora la robustez del sistema.